

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	11
Bug	Operator-blocked	1
Bug	Queued	18
Bug	Ready for testing	4
Task	In progress	12
Task	Queued	31
Task	Ready for testing	2
TOTAL		79

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at /api/v1/auth/rutrackercaptcha then /login once, and the proxy stores the resulting bb_session. The cookie-paste path and the cookies-file autoloader path were both offered and NOT chosen. THE BLOCK NARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest operator_attended boundary). AGENT PREPARED BEFORE THE HAND-OFF, so the

ATM ID	Type	Status	Severity	Description
				operator's single attempt succeeds rather than discovering a broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end, verify the session is actually PERSISTED after a successful post (not merely accepted), and verify a stored session is actually USED by a subsequent search. Honest limitation to state up front: a stored bb_session expires, this path requires repeating — that cost was accepted with the decision. This answer is recorded as consumer DATA per §11.4.35 — is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decision — prior item text follows — RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download proxy + qBitTorrent-go + Jackett + compose environment forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from mandated test-type matrix
				OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNKNOWN — INSTRUMENT THE NEXT OCCURRENCE The operator does not currently know which host produces the +0500 Auto-commit fast-forward so the three candidate answers (second Claudio session / intentional rsync job / stale job) all remain open. DECIDED ACTION: stop hunting for host this session cannot reach (§11.4.6 — remote state is not knowable without access) and instead INSTRUMENT the mechanism so the
BOB-077	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				next occurrence identifies itself. Forensic capture to add: committer identity, hostname, timezone offset, push timing, and the git remote used, recorded at Auto-commit time into a tracked forensic log. ACCEPTANCE: the next Auto-commit event yields a captured record naming its origin host — at which point this item resolves to one of the three original branches with evidence rather than a guess. This is the §11.4.101 reversible-safe move: it costs little, blocks nothing, and converts a recurring mystery into a self-identifying event. This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5)) bounded-verdict discipline applied to decisions — prior item text follows — RD2-10: Identify second host running the Auto-commit rsync/ sync mechanism (OPERATOR-DECISION) RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it RD2-13: Retroactive Fable-xhigh code review the substantive Auto-commit diffs RD2-15: Create BOB-064..067 workable items the four Lava-porting findings (closes GA-05) RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10) RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder) RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack RD2-28: Live compose bring-up + verify rustracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-078	Task	Queued	High	
BOB-080	Task	Queued	High	
BOB-082	Task	Queued	High	
BOB-085	Task	Queued	Medium	
BOB-088	Task	Queued	Medium	
BOB-090	Task	In progress	Medium	
BOB-093	Task	In progress	High	

ATM ID	Type	Status	Severity	Description
BOB-094	Task	In progress	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.90 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for export/download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one comm (canonical impl of RD2-09)
BOB-101	Task	Queued	High	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0 Go --profile go parity remains a goal but is NO LONGER a v1.0.0 release blocker. RW-10..13 move to a post-1.0 milestone and stop gating tag. Nothing advertised is removed, so §11.4.90 stays clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stands unchanged: the Go container’s Dockerfile runs ONE binary binding only :7187; nothing binds :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md’s port map already describes what the container ACTUALLY does and needs no amendment under this decision. This answer is recorded in consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — GA-19/RW-09: Is --profile go pa still a release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	In progress	Medium	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD The tunnel STA LAN-reachable — no bind address changes to 127.0.0.1. The operator explicitly chose to

ATM ID	Type	Status	Severity	Description
				preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regression guard MUST land that FAILS THE BUILD if an LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 THE CONDITION that guard exists — without this decision is unprotected and the item is N closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand maintained list; resolve real auth coverage n grep for the string ‘auth’ (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures p §11.4.107(10). This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0 OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3)
BOB-135	Bug	Ready for testing	Low	Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied /config)
BOB-137	Bug	In progress	High	Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by our spinning thread)
BOB-141	Task	Queued	Low	CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7186 — doc contradicts the Dockerfile
BOB-143	Bug	Queued	Medium	Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan scope and manufacture false BOB-126-class findings
BOB-150	Task	Queued	Low	pre_build_verification.sh invariant labels read N/44 but only 35 invariants are labelled
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc
BOB-152	Bug	Queued	Medium	Constitution sweep walks vendored third-party code: 82% of one gate's 38,291 findings come from submodules/
BOB-154	Bug	Ready for testing	Medium	Host venv and production container run different starlette versions (1.4.1 vs 1.6.0)
BOB-156	Bug	Queued	Medium	BOB-145 event-loop regression guard is load-sensitive and flaky: 8786ms under host load vs 900-1500ms ceiling calibrated on a quiet host
				OPERATOR DECISION (2026-08-26, §11.4.66): REPAIR, THEN RE-VERIFY UNTIL STABLE The warm ./start.sh path keeps running the ownership repair against a live stack, but warms and re-verifies, and repeats until a pass finds nothing new. Options not chosen: refuse the repair while the stack is up and direct the operator to –recreate; stop the stack on the warm path too (correct but ends ‘warm’); accept and document the window. WHY THIS MATTERS MORE THAN A NORMAL ITEM: this is the exact defect the whole 002-user-owned-downloads
BOB-159	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				feature exists to end. The –recreate path was already fixed to order precondition -> down -> repair -> up so the walk sees a tree no container can write. The warm path was not, so a container can create a new non-owned file AFTER the repair has passed that directory — the ownership problem leaking back in through the other door. REQUIRED BY THE CHOSEN OPTION, and it is the hard part: the loop MUST be BOUNDED, and on giving up it MUST report honestly that ownership is UNPROVEN rather than silently exiting 0 (§11.4.201(6) — a loop stops finding new files because it ran out of iterations returns the same quiet zero as a genuinely clean tree). An active download creating files faster than the walk completes the realistic non-converging case and must be named in the give-up message. Recorded as consumer DATA per §11.4.35 — the operator’s stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. — prior item text follows — Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:01:17Z Reported-By: T041 independent review (IMPORTANT-2026-08-21) partially remediated What (the report, verbatim): The –recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by three new checks in tests/unit/test_start_reload_recreate.sh, each killed by independent 1.1 mutation (11/11 RED-OK). The WALK path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records completion over a tree that is not, and those stragglers are

ATM ID	Type	Status	Severity	Description
				never repaired without a manual –force. BOUNDED, not dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits with walking (marker_is_valid), so the window requires a stale-or-absent marker AND a live stack together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-on-probe mode on scripts/ownership_repair.sh that answers has-this-scope-been-repaired without walking the tree. The existing –dry-run cannot serve: it deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding the mode requires editing ownership_repair.sh, which another stream was actively editing during this round; duplicating marker_is_valid into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misleading comment at the warm-start call site (which claimed the gate runs before any container writes) has been corrected to state this boundary honestly. Affected scope / file-scope manifest: start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path) Reproduction / context: 1. Ensure the stack is UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership_repair walks and chowns the declared tree while containers are still running and able to write into it. Acceptance criteria: A warm ./start.sh cannot complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal naming –recreate,

ATM ID	Type	Status	Severity	Description
BOB-160	Task	Queued	Medium	the stack is quiesced for the walk. Asserted behaviourally in tests/unit/test_start_reload_recreate.sh alongside the existing PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with a paired 1.1 mutation that it. Reported-Via: §11.4.202 reporting directive on 2026-08-21T19:02:48Z Reported-By: AI W (the report, verbatim): Independent §11.4.2 review (IMPORTANT-3) found two of the feature’s strongest automated checks were never executed by any runner: (1) tests/ownership/test_container_writes_owned_file — the §11.4.115 RED-turned-regression-guard for FR-011/FR-007, proven via a docker-compose.yml revert mutation — was moved of tests/integration/ (commit 58d340a, to escape an autouse fixture hang) but no runner (ci.sh test.sh, run-all-tests.sh) was ever extended to cover tests/ownership/. (2) tests/pre_build/test_.sh, the §1.1 paired-mutation meta-tests for the scripts/pre_build/check_cm_.sh gate family was executed by NOTHING — self-reported honestly in commit 04742d7’s own message (‘STILL OPEN (reported, not fixed): tests/pre_build/ is executed by NOTHING’) but never tracked as a workable item (a §11.4.197 loss-of requirements gap) and never wired into scripts/pre_build_verification.sh invariant 30, which globbed only tests/unit/test_.sh. Affected scope file-scope manifest: ci.sh; scripts/pre_build_verification.sh invariant 30 (CM-BASE UNIT-TESTS-EXECUTED) Reproduction / context: grep -rn test_container_writes_owned_files ci.sh test.sh run-all-tests.sh scripts/ returned zero runner h (only docs/specs mentioned the path); grep in scripts/pre_build_verification.sh showed invar 30’s for-loop globbing only tests/unit/test_.sh, never tests/pre_build/test_.sh. Acceptance criteria: ci.sh gains a runtime-gated pytest te

ATM ID	Type	Status	Severity	Description
				ownership/ stage (skips honestly, rc=0, when n podman/docker present); scripts/ pre_build_verification.sh invariant 30's glob covers both tests/unit/test_.sh and tests/ pre_build/test_*.sh, verified by an extracted standalone run of the invariant's exact block reporting a non-zero RAN count that includes files from both directories. Reported-Via: §11.4.202 reporting directive + on 2026-08-21T19:31:12Z Reported-By: BOB-0 remediation, residual finding What (the rep verbatim): §11.4.69 names CM-NO-FAIL-OPE SKIP as one of three mandatory pre-build gate it audits sink-side probe helpers and FAILs if code path converts an empty or unreachable response into a PASS-counting SKIP for a feat class with a sink-side probe. This project does not have it. Why this matters now rather than the abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this class from tests/e2e/test_live_stack_evidence.py — nnmclub SKIP-on-404 (already removed at 7baef2b) and a surviving sibling at test_iporrents_is_authenticated_in_search th skipped on 'not authenticated and status != success' while asserting 'treating as transient outage'. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both which are definitive product failures the product already classifies via error_type. Two fail-opens of one class, found one at a time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: th regime did not surface either, an agent reading code did. The remediation's own guards are A structural and lethal under three discriminat mutations, but they LIVE IN THE FILE THEY GUARD, so deleting that file evades them entirely. A pre-build gate is the durable home because it audits the corpus rather than one Honest boundary (§11.4.6): this item does NO
BOB-161	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
BOB-162	Task	Queued	Medium	claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CLASS has no corpus-wide detector, so the next instance in a different file is invisible again. Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard) Reproduction / context: grep -rl 'CM-NO-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Contrary to the needle: the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)(b)). Acceptance criteria: A script in scripts/pre_build/ gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converting an empty/unreachable/error response into a PASS or counting SKIP for a feature class that HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FAIL fixture (a gate with-carrier (an honest topology/geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/ §11.4.201(1). Paired §1.1 mutation makes the gate FAIL before the gate is trusted. Reported-Via: §11.4.202 reporting directive + §11.4.202 on 2026-08-21T19:35:02Z Reported-By: BOB-107 BOB-107 remediation, residual What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 30 with 3 under tests/hooks. All tests pass. But the GUARDS themselves are still

ATM ID	Type	Status	Severity	Description
				invoked by nothing. check-brief-inputs.sh is honestly conductor-run rather than automatic. brief's required-input list lives in free-form prose that the Agent tool's structured input does not expose, so an automatic prompt-scraping gate would itself be an unproven pattern-matching. Its seam is the dispatch procedure, and that is documentation + habit change, not a hook. unattributed-commit-guard.sh is different and why this item exists. Run against real history names 14 violating commits since the last tag (and 21 bare Auto-commit subjects exist across all refs). Wiring it into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, on pre-existing debt nobody can fix in the moment. That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until all 14 are attributed; (b) on-time monotone-decrease ratchet (the 11.4.135 pattern) – snapshot 14 as the baseline, the count may fall and may never rise, so day one is ground and the disease cannot spread; (c) changed-commits-only – gate only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period then escalate. Recommendation, with the reasoning rather than just the pick: (b). It is this pattern this constitution already uses for example this situation, it makes the debt visible and bounded immediately, and it cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's and this item is not closed until that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their ATTRIBUTION is absent and that nothing currently prevents the 15th. Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh;

ATM ID	Type	Status	Severity	Description
BOB-163	Bug	In progress	Medium	scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, each proven non-bluff against a no-op stub). Neither is invoked by scripts/pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage). Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing at this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an invented ratchet. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): YES — 429 IS RESPONSIVE IF Retry-After IS WELL-FORMED A 429 carrying a VALID Retry-After header counts as the sibling endpoint being RESPONSIVE. Assertion (c) of the DDoS challenge accepts 2xx OR a well-formed 429 (positive integer seconds or valid HTTP-date — PARSE not merely present/non-empty). DEGRADED remains: connection failure, timeout, any 5xx and a 429 with absent or malformed Retry-After. RESIDUAL RISK, accepted knowingly and to be stated in the script source per §11.4.6: a genuinely wedged endpoint that happens to answer 429-with-Retry-After would pass — the Retry-After parse narrows that window, it does not close it. Options (b) limiter-aware drain and (c) exempt healthz probe were both offered and NOT chosen. STILL OPEN, not covered by this decision: the detector counts 429 only, not 503 though the script header says ‘429 (or equivalent)’; counting 503 would collide with the crash detector’s 5xx tally. Decide when BOB-111 lands limiters on :7185 and :7189. The answer is recorded as consumer DATA per

ATM ID	Type	Status	Severity	Description
				§11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5)) bounded-verdict discipline applied to decisions — prior item text follows — Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:41:08Z Reported-By: BOB-114 remediation, pre-existing defect surfaced by BOB-111 landing a real limiter What (the report, verbatim): PRE-EXISTING, NOT INTRODUCED – and proven so rather than asserted: git diff shows assertion (c) is BYTE-IDENTICAL to HEAD, and the failure reproduced against the unmodified HEAD script. What changed is not the challenge but the SYSTEM. BOB-111 appears at least partially delivered, because :7187 now returns real 429s where it previously returned none. :7185 and :7189 still produce zero 429s. This is a 11.4.248-class corrosion risk and that is why it is filed rather than left as a footnote: run_all_challenges.sh now fail INTERMITTENTLY, and an intermittent red trains everyone to re-run until green, at which point a real regression is dismissed as ‘probably the flaky rate-limit one’. THE DECISION, which is an operator call under 11.4.66 and which I deliberately did NOT invoke. Does a 429 from a WORKING limiter count as the sibling endpoint being RESPONSIVE? (a) YES – a 429 proves the endpoint is alive and correctly protecting itself. Assertion (c) accepts 2xx OR 429, and only a connection failure / 5s timeout counts as degraded. Risk: a genuinely wedged endpoint that happens to answer 429s would pass. (b) NO – keep requiring 2xx, but make the challenge limiter-aware: drain or wait out the window before the sibling probe (retry after is served, so the budget is knowable rather than guessed). (c) Probe siblings on a path the limiter exempts (a healthz-class route), so the isolation question is asked without spending

ATM ID	Type	Status	Severity	Description
BOB-164	Bug		Medium	bucket. Recommendation with reasoning, not just a pick: (a) composed with a bounded liveness check – a 429 carrying a well-formed Retry-After IS evidence of a live, correctly-behaving service, and (b) makes the challenge slower and couples it to a window value that will drift. But the semantics of ‘responsive’ here are a product judgement, so the answer is recorded, not assumed. RELATED, stated as fact rather than folded in: the detector counts 429s only, not 503, though the script header says ‘4xx (or equivalent)’. Counting 503 would collide with the crash detector’s 5xx tally. Worth deciding when BOB-111 lands limiters on :7185 and :7187 now enforces a real limiter (measured live: x-ratelimit-limit: 120, x-ratelimit-remaining: 86, retry-after: 45, server: uvicorn). A full challenge run sends roughly 250 requests to :7187, so sibling probes fired during the OTH endpoints’ tiers land on an exhausted bucket and receive 429 – which assertion (c) scores as ‘endpoint degraded’. Timing-dependent: one measured PASS=5 FAIL=2; the UNMODIFIED HEAD script against the same live stack ~30s later measured PASS=7 FAIL=0 SKIP=2. Affected scope / file-scope manifest: challenges/scripts/ddos_resilience_challenge.assertion (c) cross-endpoint isolation; run_all_challenges.sh which invokes it Reproduction / context: Assertion (c) requires sibling-endpoint probe to answer ^2 (a 2xx). Acceptance criteria: The operator has answered the classification question below, the answer is recorded as consumer DATA, and assertion (c) implements it. The challenge then produces the same verdict across 3 consecutive runs against a rate-limited stack (11.4.50 deterministic consistency), and the fix ships a paired 1.1 mutation proving assertion (c) still catches a genuinely degraded sibling endpoint narrowing it must not blind it (11.4.201(1)).

ATM ID	Type	Status	Severity	Description
		In progress		Reported-Via: §11.4.202 reporting directive 11.4.202 on 2026-08-21T19:56:53Z Reported-By: BOB-11.4.202 UX-class coverage, discovered by the new axe-core suite on its first live run What (the report verbatim): This is a REAL user-facing defect, not a test-tuning artifact, and it was found by the automated regime rather than by a human squinting at the page – which is exactly the 11.4.238 posture the project is aiming for. The static-grep oracle would NOT have found it. Measured: the served root ships an empty prerender hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct. Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High. The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests/ux/ currently reports 1 failed, 16 passed; that 1 is this defect. Anyone reading a red UX suite should read it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal that it is closed. HONEST SCOPE LIMIT (11.4.6): only the dashboard landing view was scanned. The /jackett/* sub-routes and the nginx-serve-hosted :4200 route set were NOT audited. The commands to close both are recorded in docs/testing/ux_accessibility.md. So this item's nodes are a floor, not a total. Affected scope file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compile SPA as frontend/); production component CSS not test files Reproduction / context: Run tests/ux/test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21

ATM ID	Type	Status	Severity	Description
BOB-167	Bug	Queued	Medium	nodes. Measured pairs: .brand / h1 text #9d00 on background #3c3f41 = 1.43-1.62:1 (WCAG A large-text floor is 3:1); tagline #808080 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1). Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component CSS rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile to the correct mechanism never weaken the check). The existing tests/suite is the guard and already fails today, so t RED is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence per 11.4.226. WHAT. download-proxy exposes two Server-Sent-Events routes. They are the same expensive class — long-lived connections that hold a worker and a generator for their lifetime — but only one is rate-limit classed: routes.py:801 @router.get('/search/stream/{search_id}') @_rl('sse_stream') <- classed routes.py:150 @router.get('/theme/stream') async def stream_theme(...) <- NO limiter decorator _rl resolves to limiter.limit(limit_for(cls)), so /search/stream is bound to the sse_stream class while /theme/stream falls through to the application default. Measured live on the running stack: /api/v1/theme/stream reports ratelimit-limit 120. PROVENANCE + A CORRECTION WORTH KEEPING (§11.4.6). This was surfaced by the BOB-109 scaling agent, whose report framed it as: 'sse_stream_limit_decorator is defined and imported/applied nowhere ... SSE falls through to the 120/minute default: 24x less protected' That framing is WRONG and was NOT filed as given. The agent searched for one symbol NAME; the wiring uses a different mechanism (@_rl('sse_stream')), and it IS applied — to /search/stream. Verified by reading routes.py:795-806 and the _rl helper at

ATM ID	Type	Status	Severity	Description
				routes.py:46-51, with a control needle confirming other *_limit_decorator symbols show real usage in api/init.py so the zero-hit was not a blind search. The agent's MEASUREMENT (120 on /theme/stream) was correct and is what makes this a real finding; MECHANISM was not. Both halves are recorded so the next reader does not re-derive the same wrong cause. WHY IT MATTERS. An unclassified SSE endpoint is the cheapest way to pin server resources: each connection is held open, and the default class permits 120/min of them. The declared sse_stream class exists precisely because this route shape needs a tighter bound than ordinary GETs. ACCEPTANCE. (a) A decision, recorded, on whether /theme/stream belongs in the sse_stream class or genuinely warrants the default — this is a policy question not automatically a bug to patch. (b) If it belongs in sse_stream, the decorator is applied and a guard drives BOTH SSE routes and asserts each route its INTENDED class limit from x-ratelimit-limit so a future route added without a class is caught. (c) A guard that enumerates SSE-shaped routes and fails on any that carries no explicit rate-limit class — the general form, so the third SSE route does not repeat this. (d) Honest boundaries: this does not claim 120/min is exploitable in the deployment; with network_mode host and no reverse proxy every caller shares the 127.0.0.1 bucket, which BOB-111 measured and recorded separately. NOT CLAIMED. No change made. The limit values were read from headers, never driven to exhaustion — the limiter is per-IP and shared with concurrent agents on this host (§11.4.119). WHAT. scripts/run_all_challenges.sh:66 lists “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist. sed -n '66p' scripts/run_all_challenges.sh “scaling_horizontal_challenge.sh” \$ ls
BOB-168	Task	In progress	Low	

ATM ID	Type	Status	Severity	Description
				challenges/scripts/ scaling_horizontal_challenge.sh ls: cannot ac ...: No such file or directory VERIFIED independently, not taken on report — surface by the BOB-109 scaling agent and re-checked here by direct invocation. WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate- honesty defect depends entirely on how the runner treats a missing entry, and that is the first thing to determine: if it SKIPS silently, th challenge bank advertises coverage it does no have (a §11.4.266 claim-vs-reality row with n passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which tra readers to ignore it. Neither outcome is acceptable; they need different fixes. ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing ent by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history fo whether it once existed and was deleted, sinc silently-dropped challenge is the more interesting defect. (c) If the runner silently sk missing entries, that is its own finding: a miss challenge must be loud (§11.4.3 SKIP-with- reason at minimum), never absent-and-quiet SEVERITY. Low as a defect, but it sits on the challenge-coverage seam, so (c) may deserve own item. CORRECTION 2026-08-22 (§11.4.6) THIS ITEM'S PREMISE WAS FALSE, and the e was mine. Recorded openly rather than quiet edited. scaling_horizontal_challenge.sh EXIST It is at submodules/challenges/challenges/ scripts/scaling_horizontal_challenge.sh, executable, 3107 bytes, dated Aug 7. So do all other entries and the meta-runner. It was nev deleted: added to the roster at 1c959ba, the challenge itself added in the submodule at 873c0b1, and a pickaxe search across both repositories and all branches finds ZERO deletions. HOW I GOT IT WRONG.

ATM ID	Type	Status	Severity	Description
				run_all_challenges.sh reads submodules/challenges/challenges/scripts/. I checked challenges/scripts/ — a DIFFERENT directory belonging to a DIFFERENT, glob-based runner. Two similarly-named directories; I measured the wrong one. That is a §11.4.201(9) field-identity error. Worse, I wrote that it was “verified by invocation”: I had listed a directory and never run the aggregator, so the claim overstated the evidence class as well as getting the answer wrong. THE LESSON, which generalises past this item. In the SAME commit correctly caught a sibling agent making this exact class of error on BOB-167 — a zero-hit produced by searching one symbol name — and applied a control needle there. Then I omitted the needle one paragraph later on my own measurement. The distinction that would have caught it: A CONTROL NEEDLE PROVES THE INSTRUMENT CAN SEE; IT DOES NOT PROVE IS POINTED AT THE THING UNDER TEST. §11.4.201(7)(b) as commonly applied guards blindness; it does not guard mis-aiming. The needle for THIS query would have been to resolve the path the runner itself reads, not to confirm that some directory listing works. WHAT IS ACTUALLY WRONG, and it is worse than what I filed. The missing-entry path runs no challenges by definition, so it can be exercised safely. Run against a checkout where the submodule is absent — the ordinary state a clone made without –recursive — the REAL runner (byte-identity sha-asserted, not a replacement). reports: PASS: 0 FAIL: 0 SKIP: 16 TOTAL: 16 OBSERVED EXIT CODE: 0 An ENTIRELY UN-REASONABLE BANK REPORTS SUCCESS. Not one dangling entry tolerated — the whole suite silently passes without executing anything. That is §11.4.201 false-null verbatim, and it is observed-reachable rather than constructed (§11.4.115(G)). REVISIT ACCEPTANCE. (a) ANSWERED, though not as filed: the runner reports a loud, counted SKIP and then never blocks, because the exit

ATM ID	Type	Status	Severity	Description
BOB-169	Bug	In progress	Medium	expression reads only the FAIL count. (b) The dangling-entry half is VOID — nothing is dangling. (c) The real work is the exit contract roster entry the runner cannot execute must be indistinguishable from a healthy run. See decision recorded in docs/qa/BOB-168/decision_missing_vs_skip.md. RELATED, REPORTED NOT FIXED: scripts/pre_build_verification.sh:1792 carries the same SKIP/MISSING conflation one level up, for the other runner. Filed separately.
				WHAT. The §11.4.65 markdown-export mandator requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are panicked FRAGMENTS — they begin at
				WHAT. The BOB-109 scaling suite gates its two timing tests on loadavg_1m/nproc <= 0.75 — on this 8-cpu host, a 1-minute load average at or below 6.0 — and SKIPS loudly otherwise. That gate is correct and was adopted for a good reason (§11.4.201(8)): the author validated a growth-exponent threshold, then observed it FALSE-FAIL correct code at load 11.06 (baseline span 2.136), and at load 15-23 measured base spans of 2.358 against injected-cubic mutants 2.278-2.331 — the signal is smaller than the noise and the two are not separable in either direction. Gating was the honest response. THE PROBLEM. The gate has consequently NEVER RUN under its own precondition. Every recorded run today measured load 9.15-23; the independent reviewer measured 9.66 during review; the conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at loads 9.4-11.6 (span 1.882) and in three stable runs (1.759/1.832/1.848) — but never once beneath the shipped ceiling. The author stated this honestly as an inference from data rather than a run they can paste, and the reviewer confirmed the inference is sound (quiescence strictly more favourable). Sound inference is
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				still not an observed run. WHY THAT MATTE (§11.4.226). A registered, topology-present guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled quiet-window invocation, no track item — which is precisely the unexecuted-standing-guard class §11.4.226 names, where registration gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is two standing guards that, when finally executed, immediately emitted latent FAILs. ACCEPTANCE. (a) Execute both quiescence-gated timing tests on a host whose 1-minute loadavg is genuinely ≤ 0.75, — no agent fleet, no concurrent build — and capture the run as an artifact showing the resolved load reading alongside the measurement span exponent, so the precondition is provable from the artifact and not asserted. (b) Record observed GREEN polarity in the item and in the test source, replacing the current honest-inference note. (c) If the gate FAILs under genuine quiescence, that is the finding this item exists to surface, and it reopens the threshold calibration question rather than being worked around. (d) Establish a freshness contract per §11.4.226: how stale a quiescent verdict may be before the gate counts as uncovered again, so this does not silently return to never-executing. HONEST BOUNDARY. This item does not claim the gate is wrong. The evidence points the other way — quiescence is strictly more favourable than the loads where GREEN was already observed. It claims only that the run has not happened, and that an unexecuted guard can be cited as coverage. PROVENANCE. Raised as IMPORTANT-2 by the independent Fable-substrate reviewer of the BOB-109 work, and independently corroborated: the reviewer verified the SKIP is loud and prints its resolved

ATM ID	Type	Status	Severity	Description
BOB-172	Bug	Ready for testing	High	numbers, and confirmed the shipped tests do SKIP at today's load. WHAT. The rutracker SEARCH endpoint is refused by bot protection while the site itself is fully reachable. Measured unauthenticated 2026-08-21 (no cookie file read, no credential value in any artifact — §11.4.10): GET /forum/index.php -> 200, 96283 B, WHAT. download-proxy/src/api/hooks.py:96-100: def _save_hooks(hooks: list[dict[str, Any]]) -> None: try: <pre> os.makedirs(os.path.dirname(HOOKS_FILE), exist_ok=True) with open(HOOKS_FILE, 'w') as f: json.dump(hooks, f, indent=2) except Exception as e: logger.error(f'Failed to save hooks: {e}') catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally. :152 _save_hooks(hooks) :174 _save_hooks(hooks) :154 logger.info('Created hook: ...') :175 logger.info('Deleted hook: ...') : return HookResponse(hook_id=..., ...) :176 return {'message': 'Hook deleted', ...} USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is a log line nobody is watching. THIS IS THE §11.4.252 SHAPE. The path combines two dangerous capabilities from that anchor's taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks and outbound calls the system will or will not make) — so it is required to FAIL CLOSED: verify the precondition, refuse when it cannot be satisfied.</pre>
BOB-173	Bug	Ready for testing	High	numbers, and confirmed the shipped tests do SKIP at today's load. WHAT. The rutracker SEARCH endpoint is refused by bot protection while the site itself is fully reachable. Measured unauthenticated 2026-08-21 (no cookie file read, no credential value in any artifact — §11.4.10): GET /forum/index.php -> 200, 96283 B, WHAT. download-proxy/src/api/hooks.py:96-100: def _save_hooks(hooks: list[dict[str, Any]]) -> None: try: <pre> os.makedirs(os.path.dirname(HOOKS_FILE), exist_ok=True) with open(HOOKS_FILE, 'w') as f: json.dump(hooks, f, indent=2) except Exception as e: logger.error(f'Failed to save hooks: {e}') catches EVERY exception, logs, and returns None. The signature returns None, so the caller has no channel to learn the write failed. Both call sites then report success unconditionally. :152 _save_hooks(hooks) :174 _save_hooks(hooks) :154 logger.info('Created hook: ...') :175 logger.info('Deleted hook: ...') : return HookResponse(hook_id=..., ...) :176 return {'message': 'Hook deleted', ...} USER-VISIBLE CONSEQUENCE, which is why this is High and not a code-hygiene nit. A user POSTs a webhook, receives HTTP 200 and a hook_id, and the hook was never written — their automation silently never fires, and the API told them it exists. Symmetrically, a user DELETES a hook, is told 'Hook deleted', the file still holds it, and it fires again after the next restart. In both directions the product reports the opposite of what happened, and the only trace is a log line nobody is watching. THIS IS THE §11.4.252 SHAPE. The path combines two dangerous capabilities from that anchor's taxonomy — MUTATION of a shared resource (a filesystem write) and EXTERNAL SIDE EFFECT (hooks and outbound calls the system will or will not make) — so it is required to FAIL CLOSED: verify the precondition, refuse when it cannot be satisfied.</pre>

ATM ID	Type	Status	Severity	Description
				and surface the refusal. Instead it fails open, a bare ‘except Exception:’ that only logs is the exact anti-pattern §11.4.252 enumerates. At the product layer it is also a §11.4.201(6) false-nu successful write and a swallowed failure are indistinguishable to the caller. PROVENANCE Surfaced as an out-of-scope observation by the independent reviewer of the BOB-135 test-isolation work — this swallow is what turned that defect into an ‘assert 0 == 1’ mystery, because the EACCES on /config was logged and discarded while the endpoint kept returning 200. Verified here directly from source before filing (the function body and both call sites re above), not taken on report. Recorded as a §11.4.238 discovery-channel escape: found by agent reading code during an unrelated investigation, not by the automated QA regime — the coverage gap is a defect of equal standard to the defect itself. ACCEPTANCE. (a) _save_hooks propagates failure — raise, or return a status the callers must consume. (b) Both endpoints translate a persistence failure into an HTTP error (500-class), never a success body; a create that did not persist must not return a hook_id. (c) A test drives each endpoint with the hooks file unwritable (read-only dir a patched open raising OSError) and asserts a non-2xx status AND that a subsequent GET does not list the phantom hook — assert on the user-observable outcome, not on the log line. (d) Paired §1.1 mutation: restore the swallow; the test must FAIL. (e) Audit the same file for sibling swallows — this is a pattern, and one instance rarely alone. (f) Honest boundary: this does not claim the write currently fails in production; it claims that WHEN it fails the user is told the opposite, and the BOB-135 investigation shows it does fail in at least one real environment. NOT CLAIMED. No change made. No assessment of how often the write fails in the operator’s deployment. RECORDING A SHELL ERROR ON MY OWN (§11.4.6): the first version of this

ATM ID	Type	Status	Severity	Description
BOB-174	Bug	In progress	High	description was written with the anti-pattern snippet inside backticks in a double-quoted s argument, so the shell ran it as command substitution and the text was replaced by nothing — the stored description read ‘and is the exact anti-pattern’. This is the SECOND time this session that backticks-inside-double-quotes has corrupted content (the first mangled a commit message). Fixed here by editing through a Python client with no shell quoting in the patch. Noted because a silently-truncated defect description is exactly the kind of quiet corruption §11.4.201(7)(c) warns about — the path is part of the instrument, and it failed without erroring. WHAT. A corrupt hooks file is silently indistinguishable from “no hooks configured” and the next create then DESTROYS every existing hook while returning HTTP 200. Three defects on one path, filed together because fix any one alone leaves the data loss reachable. — download-proxy/src/api/hooks.py:104-112. _load_hooks wraps the read in except Exception: logger.error(...) and falls through to return []. A truncated or malformed hooks.json is therefore reported to every caller as an empty, healthy hook list. Confirmed at source. A2 — the consequence, and the reason this is High. create_hook loads, appends, saves. Given a corrupt file that load turns into [], the save writes a one-element list over the top. Every previously-configured hook is gone. Measured by the implementing agent against the ALREADY-FIXED tree, so this survives the BOB-173 write-failure fix: BEFORE on disk : [‘prod-hook-0’, ‘prod-hook-1’, ‘prod-hook-2’] C reports : 200 {‘hooks’: [], ‘count’: 0} <- claims ZERO hooks configured DELETE reports : 404 {‘detail’: ‘Hook not found’} <- for a hook that in the file POST reports : 200 hook_id=3c8a5930-... AFTER on disk : [‘3c8a5930-...’] Three prod hooks destroyed,

ATM ID	Type	Status	Severity	Description
				HTTP 200 throughout, nothing surfaced to the user. A5 — <code>_save_hooks</code> writes with a plain <code>open(path, "w")</code>. A crash, <code>ENOSPC</code>, or a kill m write truncates the file in place. That is precise the corruption A1 then reads as “no hooks” and A2 overwrites. The same codebase already has the correct pattern: <code>theme_state.py:115-126</code> uses <code>tmp-file + os.replace</code>. WHY THE THREE ARE CORRUPT ITEM. A5 manufactures the corrupt file, A1 misreads it as empty, A2 destroys the content. Fixing only A1 leaves truncation reachable; fixing only A5 leaves an existing corrupt file as a data-loss trigger; fixing only A2 leaves the API lying about what is configured. The chain is the defect. DELIBERATELY NOT FIXED UNDER BOB-173, and the reasoning is sound. The implementing agent identified all three while auditing sibling swallows as that item requires and declined to fix them in the same change because: they change GET semantics on an endpoint the frontend consumes (200 -> 5xx) they need a CORRUPT-file reproduction rather than the UNWRITABLE-file one BOB-173 built and they need a design decision that is genuinely not obvious — a MISSING hooks file legitimately means “no hooks configured”, while a CORRUPT one does not, and today’s code cannot tell those apart. Expanding BOB-173’s scope to cover them would have meant shipping that decision unexamined. ACCEPTANCE. (a) Distinguish MISSING from CORRUPT: a missing file remains an empty list; a corrupt one is an error, never silently []. (b) Decide and RECORD what GET does on corruption — 5xx, or 200 with an explicit degraded marker the frontend can render. This is the design decision, and it should be stated rather than inferred from whatever the patch happens to do. (c) create/delete MUST NOT overwrite a file they could not parse — refuse, do not clobber (§11.4.252: mutation prevents external side effect must fail closed). (d) Make the write atomic via <code>tmp + os.replace</code>, reusing <code>theme_state.py</code>’s existing pattern rather than

ATM ID	Type	Status	Severity	Description
				inventing it (§11.4.28). (e) Tests asserting the USER-OBSERVABLE outcome, not log lines: see a corrupt file, assert GET does not claim zero hooks, assert a create refuses rather than destroying, and assert the file still holds the original hooks afterwards. (f) Paired §1.1 mutation per guard. (g) NEGATIVE CONTROL (§11.4.201(1)): a MISSING file must still yield an empty list and a working create; a VALID file must behave exactly as today. A fix that makes every load fail closed by failing always is not a fix. A3, RECORDED SEPARATELY, NOT PART OF THIS CHAIN. VALID_EVENTS and HookEventType are two sources of truth for a closed set. Verified identical today, unguarded against drift: if they diverge a hook registers successfully and then silently never fires. On an assertion pinning them would close it. NOT CLAIMED. No change made. The BOB-173 write failure fix is real and orthogonal — it makes a FAILED write honest; it does nothing about a SUCCESSFUL write of wrong data derived from a misread file. WHAT. The workable-items update seam guards the status-location invariant in one direction only. After BOB-166, update --status <terminal> on an Issues-located row is refused. The mirror is still open: update --location Fixed --status <non-terminal> exits 0 and creates a row that update's own validator immediately rejects. Verified empirically by the independent reviewer of BOB-166: update --location Fixed --status 'In progress' -> exit 0 validate -> FAILS naming the row (check (f), fixedLocationNonTerminalStatus) So the command can still mint a state its own validator call refuses. That is the §11.4.196(F) shape at the seam layer: the invariant is CONFIGURED in validate but not ENFORCED at every write path that can violate it. WHY IT WAS DELIBERATELY LEFT OPEN IN BOB-166, and why that was right. Three reasons, all checked rather than asserted.
BOB-175	Task	Queued	Low	

ATM ID	Type	Status	Severity	Description
				(1) the real corpus has ZERO instances of this direction against TEN of the direction BOB-16 closed — the asymmetry in the data matched asymmetry in the code; (2) detective coverage ALREADY existed and demonstrably fires, so unlike BOB-166's direction this cannot rot silently; (3) and the load-bearing one — reopenCmd documents and SANCTIONS a transient Fixed-plus-non-terminal state via its location Fixed operator override. A naive preventive guard at the update seam would collide with that override's semantics. Closing this therefore requires a design decision about how the two interact, which is a different question from the one BOB-166 was scoped to answer, and expanding that item would have meant shipping the decision unexamined. ACCEPTANCE. (a) Decide and RECORD how a preventive guard on this direction coexists with reopenCmd's sanctioned override — this is the actual work, and the answer should be written down rather than inferred from whatever the patch does. Candidates worth weighing: exempt the reopen path explicitly, require an override flag on update mirroring reopen's, or leave the direction detective-only with the rationale recorded so the asymmetry is a decision rather than an oversight. (b) If a guard lands, it reuses terminalStatuses() — BOB-166 established a predicate source specifically so the two directions cannot drift. (c) Paired §1.1 mutation (d) NEGATIVE CONTROL (§11.4.201(1)), and it's the sharp one here: the sanctioned reopen path MUST still work. A guard that breaks reopenCmd's documented override is a false-positive refusal, and worse than the gap it closes, because it breaks a workflow operators are told to use. HONEST BOUNDARY. This is not a live data defect. Zero corpus instances, and the detective gate catches it at the next validate. It's filed so a scoped, deliberate omission stays visible as a tracked decision instead of living only in a code comment where the next reader

ATM ID	Type	Status	Severity	Description
				will mistake it for an oversight (§11.4.197: a started thread reaches a documented termin state, never quiet abandonment). PROVENANCE Raised by the implementing agent of BOB-160: a known asymmetry it deliberately did not check independently verified and endorsed as an acceptable scope boundary by that item's reviewer, on the condition that it become a tracked item rather than a comment. This is the item. WHAT. <code>_get_enabled_trackers()</code> in <code>download-proxy/src/merge_service/search.py</code> gates <code>rutracker</code> on <code>RUTRACKER_USERNAME</code> and <code>RUTRACKER_PASSWORD</code> only. It never consumes <code>RUTRACKER_COOKIES</code>. So an operator configured with cookies alone — no username, no password — never has <code>rutracker</code> enabled at all, and the tracker is silently absent from even fan-out rather than failing visibly. WHY THAT IS INCOHERENT WITH THE REST OF THE CODEBASE <code>_search_rutracker</code> treats <code>COOKIES</code> as the PREFERRED auth path, not a fallback. And the <code>nnmclub</code> sibling DOES check its own <code>*_COOKIES</code> variable. So the same codebase holds three positions at once: cookies preferred at the search site, cookies ignored at the enablement gate, and cookies honoured for a sibling tracker. WHY IT MATTERS MORE THAN IT LOOKS. CLAUDE.md documents a standing operator mandate (2026-08-15) that per-tracker Netscape cookies files at <code>\${TRACKER_COOKIE_DIR}/cookies_<tracker>.txt</code> are auto-loaded into <code>.env</code> as <code><TRACKER>_COOKIES</code> before every <code>boba-setup</code>, <code>restart</code>, <code>install Stage 6</code> and <code>start.sh</code> boot — and <code>rutracker</code> is named in that set. So the SANCTIONED, documented configuration pattern for this tracker produces exactly the state this gate ignores. An operator who follows the documented instructions gets a tracker that never runs, with no error to explain it. FAILURE MODE. Silent absence, not a visible failure — same §11.4.201(6) false-null shape as BOB-170.
BOB-176	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				one layer earlier. BOB-172 fixes a tracker that RAN and was REFUSED being reported as empty; this is a tracker that never ran at all, and the merged result simply has one fewer contributor with nothing to indicate it. PROVENANCE. Found by the BOB-172 implementing agent while tracing the enablement path to locate the status-handling defect. Recorded here rather than fixed in the change: it is a separate defect on a separate seam with its own oracle, and folding it in would have widened BOB-172 past its captured evidence. ACCEPTANCE. (a) The enablement path honours RUTRACKER_COOKIES as an independently sufficient credential, matching what _search_rutracker already prefers and what the nnmclub sibling already does. (b) A test asserting that a cookies-only configuration ENABLES rutracker, driving the real _get_enabled_trackers rather than a replica. (c) Paired §1.1 mutation restoring the username/password-only gate — the test must go red. (d) NEGATIVE CONTROL (§11.4.201(1)): a configuration with NO credentials at all must still leave rutracker DISABLED. A fix that enables an unauthenticated tracker is worse than the gap, because it produces failing searches instead of absent ones. (e) Audit the other trackers' gates for the same asymmetry rather than fixing only the one that was noticed — three positions in one codebase suggests nobody has checked them together (§11.4.118). HONEST BOUNDARY. Not verified against a live cookies-only deployment; read from source by the BOB-172 agent and recorded on its report. The reading is specific and checkable, but whoever takes this should confirm by invocation before relying on it. BOB-177 Bug Queue High WHAT: BOB-172 wired an identical 4-line HTTP-refusal guard at five private-tracker fetch sites in download-proxy/src/merge_service/search.py (rutracker cookie :1468, rutracker credential :1468, kinozal :1648,

ATM ID	Type	Status	Severity	Description
				nnmclub :1761, iptorrents :1934). Only the rutracker COOKIE path is exercised by tests. EVIDENCE (reviewer-authored mutation R1, 11.4.194(6)(d), during the BOB-172 independent review): deleting ONLY the kinozal guard wiring (search.py:1655-1658) while leaving the classifier intact left the full merge_service suite at 883 passed, ZERO failures. The suite cannot see four of the five sites. FAILING SCENARIO: refactor drops or subtly breaks the wiring at kinozal, nnmclub, iptorrents, or rutracker-credential. A 403 at that site silently folds back to status=empty with error=None -- the exact BOB-172 signature -- and no test reddens. FIX DIRECTION (reviewer preferred): collapse the five duplicated wirings into one shared helper e.g. _check_search_response(tracker_name, status, body) -> bool, so there is ONE copy to test and a sixth site cannot be added unguarded (11.4.251 byte-identical-fork extraction). Alternative: parametrise the guard tests across all five sites with per-site stub sessions. ACCEPTANCE: mutating the wiring at ANY of five sites reddens at least one test. Prove it by running the same R1 deletion at each site in turn. BOB-178 Bug Queued Medium WHAT: download-proxy/src/merge_service/search.py:1638-1640 handles a failed kinozal LOGIN response with if login_resp.status not (200, 301, 302): logger.error(...); return [] -- and sets NO diagnostic. FAILING SCENARIO: Cloudflare returns 403 on takelogin.php. The kinozal chip reads status=empty, error=None. That is the BOB-172 signature exactly: a refusal reported to the user as an empty result set. CONTRAST establishing this is an oversight, not a design choice: the rutracker and nnmclub login failures DO set diagnostics (upstream_captcha / auth_failure), and the iptorrents login failure falls through to the search fetch where BOB-172's new guard catches it. Kinozal is the one leg with neither. FIX DIRECTION: stash

ATM ID	Type	Status	Severity	Description
				_classify_upstream_http_status(login_resp.status) before the early return, or an auth_failure diag for the status-in-(200,301,302)-but-no-cookie case. ACCEPTANCE: a stubbed 403 on the kind login endpoint produces a non-None error on the kinozal chip, with a RED captured against the pre-fix code first (11.4.115). BOB-179 Task Queued Medium WHAT: the BOB-1 independent review demonstrated two further paths that still report a refusing or unreachable tracker as an empty result. Both are PRE-EXISTING and neither is a regression from BOB-172, but the fix evidence log presents the five-site coverage without stating the boundary (11.4.194(5) requires un-analysed dimensions explicit gaps, never silently assumed safe). G. A -- exception before resp.status is read. Reviewer probe B, captured: a connection-refused rutracker yields status=empty, error=None, http_status=None, metadata.errors=[], metadata.status=complete. Each _search_* method swallows exceptions (except Exception: logger.error; return result) before _search_one's error handling can see them, so a tracker that is DOWN is indistinguishable from one that is genuinely empty. GAP B -- soft refusal at HTTP 200. Reviewer probe A, captured: _classify_upstream_http_status(200, <cf-chl body>) returns None, which is CORRECT by design (a 2xx is a usable response; body-marker triggering would risk the over-fire the negative controls exist to prevent). But all five search GETs use aiohttp's default allow_redirects=True so a session-expiry 302 -> login-page-200 chain parses to zero rows and reports empty. FIX DIRECTION: Gap A needs the exception to read a diagnostic rather than being swallowed. Gap B needs a DISTINCT detector (final-URL check of login-form marker) with its own RED -- explicit NOT a widening of the status-code trigger. ACCEPTANCE: both gaps closed with their own REDs, or explicitly closed per 11.4.112 with

ATM ID	Type	Status	Severity	Description
				evidence. Immediate sub-task: append a state gaps paragraph to docs/qa/BOB-172/fix_evidence_20260822.log. BOB-180 Bug Queued Low WHAT: download-proxy/src/routes.py:727-745 sets status=captcha_required with a hardcoded message naming RuTracker 'RuTracker requires CAPTCHA. Use /api/v1/auth/rutrackercaptcha'. FAILING SCENARIO: a whois search returns zero results and the captcha-flavoured diagnostic came from NNMClub's Turnstile, not RuTracker. The user is told to visit a RuTracker captcha endpoint for an NNMClub problem. SEVERITY BOUNDED: the full error and tracker_stats travel in the same response payload, so the truth is present and a client that reads them is not misled -- only the headline is wrong. The branch was revived by BOB-172's error propagation (correctly: suppressing that propagation would recreate the same false-name one layer up, 11.4.247) and is already covered in the routes layer by tests/unit/api_layer/test_routes_coverage.py:782. FIX DIRECTION: derive the message from the tracker(s) that actually erred rather than hardcoding one. SCOPE NOTE: api/ was owned by a sibling stream during BOB-172; the author was correct not to touch it. ACCEPTANCE: an NNMClub-only captcha diagnostic on a zero-result search produces a headline naming NNMClub. BOB-182 Task In progress Medium **OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP THE MONOTONE RATCHET** §11.4.224(E) brownfield adoption answer, recorded as consumer DATA: the adoption model for CM-EXPORT-CHARSET-VALID is the MONOTONE-DECREASE RATCHET. The count may only go down, never up. Immediate hard floor, changed-code-only-with-deadline, and per-corpus phase-in were all offered and NOT chosen. This closes half of BOB-182 — the half that was genuinely the operator's. THE REMAINING HALF IS A REAL DEFECT STILL OWED: BASELINE is a hardcode

ATM ID	Type	Status	Severity	Description
				constant, so the ratchet does not actually ratchet — on improvement the gate PASSES and PRINTS the value to lower it to, but nothing lowers it, and the gate keeps permitting regression back to the original count after the corpus heals. The gate must NOT collapse §11.4.249 role separation: a gate that writes its own threshold during a pre-build run becomes a PRODUCER as well as a GATE. Acceptable shapes include a persisted baseline the gate READS but never writes with lowering via a separate explicitly-invoked command, or a gate that FAILS loudly when the live count is below baseline so drift becomes actionable refusal. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). --- prior item text follows --- WHAT: CM-EXPORT-CHARSET_VALID (pre-build invariant 50) adopts its 301 pre-existing violations via a monotone-decreasing ratchet rather than a hard floor. Two things are owed. (1) THE ADOPTION DECISION IS THE OPERATOR'S, NOT THE SCRIPT'S. 11.4.224(E) requires the brownfield adoption question be SURFACED to the operator and the answer recorded as consumer DATA -- never an invented ratchet. What exists today is a head comment plus a BOBA_EXPORT_CHARSET_BASELINE override that DOCUMENTS a decision the agent made, does not RECORD an answer the operator gave. The independent reviewer judged the choice defensible and would not reverse it (a hard floor on 301 violations makes the build unreachable which 11.4.234 forbids; 11.4.101 favours the reversible option) but correctly held that it is yet compliant. Options for the operator: immediate hard floor (BASELINE=0) / keep the monotone ratchet / per-corpus phase-in /

ATM ID	Type	Status	Severity	Description
				changed-code-only with a scheduled full-corp deadline. (2) THE RATCHET DOES NOT ACTUALLY RATCHET. BASELINE is a hardcoded constant. On improvement the gate PASSES and PRINTS the value to lower it to, but nothing lowers it, so the gate keeps permitting regression all the way back to 301 after the corpus heals. The header now says tightening MANUAL rather than claiming automatic behaviour that does not exist (11.4.6), but the honest statement is a stopgap, not the fix. WHY IT WAS NOT BUILT WITH THE FIX: a gate that writes its own threshold during a pre-build r becomes a PRODUCER as well as a GATE (11.4.249 role separation), and that is a design change the operator should approve rather than receive as a side effect. ACCEPTANCE: the operator's adoption answer recorded as consumer DATA; and either a persisted baseline the gate lowers and never raises (with the role separation question answered), or an explicit decision that manual tightening is acceptable BOB-184 Task Queued Medium Threat icon-glyph controls (.bridge-retry, .theme-toggle, .caret) render text made of non-BMP code points. axe-core declines to decide their contrast reporting them under the incomplete channel with messageKey nonBmp and resolving NEITHER fgColor NOR bgColor, so the Python recomputation in docs/qa/BOB-164/axe_contrast_scan.py cannot decide either: 40 such nodes across 16 palette x mode combinations. BOB-164 round 2 stopped them from being scored as silent passes by naming them GLYPH_UNMEASURED, printing them on every run and fencing them, so any NEW unmeasured glyph node blocks. What remains genuinely unproven is their contrast itself: as non-text user-interface components they owe 3:1 under WCAG SC 1.4.11, and no oracle in this repo measures that today. Acceptance: a measurement path that resolves the effective foreground and backdrop for glyph controls

ATM ID	Type	Status	Severity	Description
				example reading <code>getComputedStyle</code> colour against the composited backdrop, or replacing the glyphs with inline SVG carrying explicit font tokens), each of the three controls shown to clear 3:1 in all 16 palette x mode combinations and the GLYPH_UNMEASURED fence shrunk exactly the nodes then proven. BOB-185 Task Queued Medium docs/qa/BOB-164/ <code>axe_contrast_scan.py</code> scans a static dist with a backend, so every node whose existence depends on fetched data is absent from the page it measures: the results table renders empty, so <code>.type-badge</code> and <code>.quality-badge</code> never appear and the hooks list renders empty, so <code>.status</code> never appear. That is precisely why the BOB-round-1 regression went unseen in a rendered DOM even though it was scanned in all 16 themes, and it is why round 2 had to close the class in the arithmetic oracle instead. Those nodes are now covered by <code>frontend/src/app/models/style-contrast.spec.ts</code>, which extracts declared foreground-on-fill pairs from the stylesheets, but that is an ARITHMETIC claim about declared colour pairs, not a rendered-pixel one: it cannot see opacity applied by an ancestor, a composited backdrop, or a cascade this repo's static extractor does not model. Acceptance: the scan drives the dashboard with seeded fixture data (a stub backend, a route fixture, or an injected component harness) so the badge and status nodes are present in the DOM, axe measures them directly, and a control needle proves the rendered oracle sees a seeded sub-floor badge before any clean result from believed. BOB-187 Bug In progress — <code>scripts/ownership_precondition.sh</code> consumes the SAME unreviewed .env-driven scope as <code>ownership_repair.sh</code> and creates probe files inside it, but it is NOT fenced. T028 remediation added <code>ownership_path_fence()</code> to <code>scripts/lib/ownership.sh</code> and wired it into <code>ownership_repair.sh</code>, closing the path-escape class there; the precondition was outside that

ATM ID	Type	Status	Severity	Description
				agents file scope and still accepts whatever config/owned_paths.yaml expands to. Blast radius is lower than a recursive chown (it won't probe files rather than mutating ownership of an existing tree) but the INPUT is identical and equally unreviewed, so the same QBITTORRENT_DATA_DIR value that would have driven a filesystem-wide chown will drive probe-file creation at an arbitrary location. Note the escape is empirically confirmed for the repair path, not merely reasoned: during T02 remediation a probe declaring the verbatim shipped entry with QBITTORRENT_DATA_DIR hung the suite, and ps showed the real artifact executing "find / (! -uid 1000 -o ! -gid 1000) -printf ..." before it was killed by verified pid. Acceptance: ownership_precondition.sh calls SAME ownership_path_fence() predicate from scripts/lib/ownership.sh (never a second dial 11.4.251), refuses fail-closed on a rejected entry and ships a paired 1.1 mutation proving the refusal fires PLUS a negative control proving six live shipped entries still ACCEPT so the fix is not a 11.4.201(1) false-positive refusal. Discovered out-of-band during T028 remediation and reported by the agent as needing tracking, so it is also a 11.4.238 covered escape. BOB-188 Bug In progress — **OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNTRACK THE BIN. — BUILD ON DEMAND** §11.4.30 answer, recorded as consumer DATA: the compiled `workable-items` binary is NOT to be git-tracked 'Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned' applies without exception here. Keeping it tracked with only invariant-52 fingerprint gate, and keeping it tracked with a commit-time rebuild, were both offered and NOT chosen. REQUIRED END STATE the gate builds from source, or REFUSES HONESTLY when the Go toolchain is absent (§11.4.201(11) — probe the artifact through it

ATM ID	Type	Status	Severity	Description
				real invocation path, never fake a pass, never silently fall back to a stale binary). Staleness becomes structurally impossible rather than merely detected. ACCEPTED COST: a fresh clo needs a Go toolchain before invariant 17 can run — already true for the qBitTorrent-go backend, so this adds no new host requirements. The invariant-52 fingerprint gate remains useful as defence-in-depth for as long as any binary exists on disk. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). --- prior item text follows --- pre_build_verification.sh invariant 17 (CM-WORKABLE-ITEMS-VALIDATE) resolves its binary through the candidate loop :534, whose FIRST entry is constitution/scripts/workable-items/bin/workable-items. That file is GIT-TRACKED (md5 17644a248363, identical to bin/workable-items-linux) and executable, so WINS resolution over the current untracked sibling constitution/scripts/workable-items/workable-items (md5 43376a6d0184). The tracked binary is STALE: it does not contain t guards its own source now has. MEASURED, needle-proven, 2026-08-25. String presence in the tracked binary: "refusing to set terminal status" -> 0; "Issues-location item has TERMINAL status" -> 0; control needle from the SAME updateCmd, "at least one mutable field flag is required" -> 2 (so the instrument sees through that path and the zeros are real absences, not blind read). The untracked sibling returns 1 / 2 for the same three queries. A check whose message string is absent from the binary can fire. RUNTIME PROOF: the same injected violation (a row set to terminal status while current_location=Issues) run through both binaries -> stale reports 1 violation (only the

ATM ID	Type	Status	Severity	Description
				body_md desync class), current reports 2 including the location-status class verbatim. ' is exactly why the ten BOB-166 rows (BOB-087/129/131/144/145/146/148/153/155/15 survived undetected: the gate that was supposed to catch them was running a binary structure incapable of seeing them. The 11.4.108 SOURCE >ARTIFACT gap: source correct, artifact stale, every gate reading the artifact reports green. This is the THIRD instance of that class found one session, alongside BOB-169 (a charset gate that never opened a file) and BOB-183 (a service bundle five days older than its sources). COMPOUNDING: the comment at :519-523 directly above the loop asserts "The naive bin/workable-items path never existed in this checkout (bin/ is a gitignored local build-output dir nothing ever populated)". That statement now FALSE - bin/ holds two tracked binaries. stale comment asserting the absence of the v file that now shadows resolution is how this stayed invisible. Also an 11.4.30 question the operator owns: bin/workable-items and bin/workable-items-linux are VERSIONED BUILD ARTIFACTS in a constitution submodule that inherited by reference by every consumer, so every consumer inherits whichever binary was last committed. Flagged independently by two separate agents this session. ACCEPTANCE: (a) resolution must never prefer a stale artifact over a current one - either the tracked binaries are removed and resolution falls through to build-on-demand, or a freshness check refuses binary older than its sources (see the BOB-18 fingerprint gate for a working pattern); (b) the false comment at :519-523 corrected; (c) a pair 1.1 mutation proving the new refusal fires, plus a negative control proving a genuinely fresh binary still passes so the fix is not an 11.4.201 false-positive refusal; (d) the 11.4.30 tracked-binary decision recorded as operator DATA. Discovered out-of-band during BOB-166 remediation - a 11.4.238 coverage escape in it

ATM ID	Type	Status	Severity	Description
				own right. BOB-189 Bug Queued — WHAT: the fail-open scanner's shape-(A2) heuristic cannot distinguish 'return False' meaning PROCEED-AS-IF-FINE from 'return False' meaning REFUSE. It flags six textbook CLOSED guards as fail-open defects: <code>_is_safe_fetch_url</code> (x3), <code>_qbit_add_succeeded</code> and the hooks path-boundary guard. INDEPENDENTLY VERIFIED 2026-08-25 by the conductor rather than taken on the triage agent's word: <code>download-proxy/src/api/routes.py:1122</code> defines <code>_is_safe_fetch_url</code> returning bool, and its only call site at :1476 reads 'if not <code>_is_safe_fetch_url(url)</code>: <code>logger.warning(Refusing SSRF-unsafe download URL (non-public target); skipping); continue</code>' False return REFUSES the URL. WHY THIS MATTERS: per §11.4.201(1) a false-positive refusal is a FAIL-bluff of equal severity to a false negative pass, and here the consequence is worse than noise - acting on the finding would mean making the SSRF guard stop returning False, i.e. deleting an SSRF protection to satisfy the gate. A gate that instructs you to remove a security control is actively dangerous, not merely imprecise. REPRO: run the fail-open scan; observe six hits; read each call site. ACCEPTANCE: the scanner distinguishes refusal shaped from proceed-shaped False returns (code site-aware, or an audited waiver list with per entry justification), the six drop out, AND a golden-FALSE fixture containing a real fail-closed guard is added so the discrimination is itself falsifiable per §1.1. BOB-191 Bug Queued — WHAT: the §11.4.252 fail-open scan reports 0 hits for qBitTorrent-go, and the zero is NOT evidence. The triage agent ran control needles through the scanner's own pass per §11.4.201(7)(b): a Python 'except Exception' pass' needle was SEEN, a TypeScript 'catch (error)' needle was SEEN (so frontend/src's zero IS a needle zero), but a Go empty-'if err != nil {}' needle was NOT SEEN. An instrument that cannot see the

ATM ID	Type	Status	Severity	Description
				idiom returns the same quiet zero as a clean tree, and only one of those is honest. DISTINGUISH FROM BOB-189, deliberately not merged with per §11.4.214: BOB-189 is a false MATCH (fail-closed guards reported as fail-open); this is a false NULL (an entire language unscanned). Same scanner, opposite failure directions, different fixes - merging them would hide one behind the other. IMPACT: Go is the language qbittorrent-proxy-go and boba-jackett (port 7189, which owns encrypted tracker credentials), so the unscanned surface is exactly where §11.4.252's credential-plus-mutation combination is most likely. Nobody has assessed it; the dashboard says clean. ACCEPTANCE: the scanner grows a Go arm covering the empty-block and swallowed-error idioms, its needle SEEN through the real path, qBitTorrent-go's result is re-derived, and every finding is triaged as this Python/TS pass was. Until then qBitTorrent-go's fail-open posture is UNKNOWN and must be reported as UNKNOWN, never a === ROUND-2 INVESTIGATION, MEASURED 2026-08-26 — THE FILED TITLE WAS TOO NARROW (§11.4.6) === The live scanner is constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh, driven per-root by invariant 39 at scripts/pre_build_verification.sh:1481-1544. (scripts/pre_build/check_cm_no_fail_open_skip.sh has NEVER existed on any branch — it exists only in an uncommitted worktree. Any citation of the path is citing a file that is not there.) IT IS A WEAK-MATCHER DEFECT, NOT A SCOPE HOLDING. Line :243 sets exts="py go rs c cc cpp h hpp java cs js ts jsx tsx php rb" — .go IS enumerated. B line :537 gates the Python ast analyser on `cat "\$f" in *.py), so every non-Python file falls through to just two text matchers (:772 emptycatch (...) {, :794 credential=x "literal"). Go has nocatchkeyword, so neither matcher CAN fire on Go by construction. NEEDLE TEST, BOTH ARMS (planted / rc /

ATM ID	Type	Status	Severity	Description
				result): Python 2 / 1 / SEEN · TypeScript 1 / SEEN · Java 1 / 1 / SEEN · **Go 6 / 0 / NOT SEEN – PASS** · Rust 2 / 0 / NOT SEEN · Ruby 1 / 0 / NOT SEEN · C 2 / 0 / NOT SEEN Go is ONE OF FIVE BLIND EXTENSIONS, not a special case. The Go needle is real code: build rc=0, go vet rc=0, gofmt -e clean. DECISIVE DISCRIMINATOR – same bytes, extension changed: .zig (NOT enumerated) – "SKIP – topology_unsupported"; .go (enumerated) → "PASS". The gate ALREADY OW an honest-blindness mechanism and it works correctly; Go routes around it precisely BEING ENUMERATED. A scope hole announces itself; this matcher hole prints green. Th makes it the worse defect and is why the corrected title leads with it. QUANTIFIED SURFACE (exclusions stated, §11.4.224(E)): using the scanner's OWN prune list and nothing added – 106 .go files / 20,389 LOC (52 production files / 6,858 LOC). WITH an WITHOUT exclusions the count is 106 == 106 no vendored tree exists, so no unstated exclusion could manufacture a clean number qBitTorrent-go holds 106 blind files and 0 analysable files – 100% blind – yet invariably 39 counts it among "5 first-party source root(s)" reported clean. DEFECTS THE BLINDNESS HID → filed as BOB-204 (credentials DELETE returns unconditional 204 while the .env plaintext delete error is discarded plus the env_write_failed_db_rolled_back of asserting an unconfirmed rollback). SEPARATE SCOPE HOLE → filed as BOB-205 (cmd/boba-ct 947 LOC orchestrator, absent from DANGER_ROOTS entirely). Distinct-but-similar per §11.4.214, deliberately not merged. GOLDEN-FALSE FIXTURE FOR ANY FUTURE GO ARM internal/jackettapi/auth_middleware.go:45- is correctly fail-CLOSED (constant-time compare, 401+return) – a Go arm MUST NOT fail it. LOAD-BEARING §11.4.250 PREDICTION: bolting a naive Go regex arm on will

ATM ID	Type	Status	Severity	Description
				IMMEDIATELY manufacture BOB-189-class fals positives in Go <code>-if err != nil { return false }</code> in a Go validator is fail-CLOSED. Fixing t as layer N+1 reproduces BOB-189 in a new language. THEREFORE the remediation order fixed: (1) analyser REGISTRY with an UNANALYSED verdict FIRST – extension→analy map, unmapped extension reports UNANALYSED never silent PASS, which converts the fals null into an honest gap and covers rust/r C too; (2) only then a Go arm, call-site-aware from day one (go/ast, or adopt errcheck/staticcheck rather than hand-roll regex), shipped with the auth_middleware golden-FALSE. ONE PRIMITIVE, TWO SYMPTOMS the scanner classifies by local syntactic shape and never consults semantic role – BOB-189 ignores the CALLER's context (fals match), BOB-191 searches for a shape from DIFFERENT language family (false null). Th gate's own header already concedes the primitive ("requires real data-flow analysis this gate CANNOT honestly claim"); what is missing is propagating that concession into PER-LANGUAGE honesty. Keep two tracker items. INSTRUMENT BUG RECORDED, NOT HIDDEN (§11.4.201(12)): the investigation's first verdict extraction used <code>grep -oE 'PASS FAIL SKIP'</code>, which matched FAIL inside the gate's own name (...FAIL-CLOSED) and mislabelled three PASSes. Re-run keyed on exit code; t table above is the corrected run. EVIDENCE docs/qa/BOB-191/scanner_blindness_investigation_20260826.m (255 lines, §11.4.44 header). No source, gate, or config modified by the investigation. BOB-192 Task Queued – WHAT: the BOB-161 gate lands with 6 re fail-open skips RATCHETED rather than fixe Ratcheting is the constitution's named brownfield default (§11.4.135/§11.4.224(E) and this repo's own precedent, so the choi is correct - but the remediation it defers

ATM ID	Type	Status	Severity	Description
				real work that must be owned somewhere. WHEN THIS ITEM EXISTS: the gate's own header asserted the 6 were 'TRACKED SEPARATELY (§11.4.197)' while no tracker row existed. The §11.4.209 independent review verified absence and raised it as IMPORTANT-5, noting that without a row those findings are precisely the parked-unverified debt class §11.4.226(4) names - the population an operator samples and finds broken. A prose claim of being tracked is not tracking. WHEN EACH NEEDS: a skip that fires on evidence host ANSWERED must either classify the response and FAIL on it, or take a §11.4.6 reason that is honestly derivable from the environment rather than from the response verified against the live stack, not asserted. Two of the six sit under '# allow skip:' markers at tests/unit/test_tracker_auth_live.py:105 and :108, where the reviewer confirmed genuinely are fail-open, so that marker must not be treated as an absolute. ACCEPTANCE: all 6 remediated with RED-first evidence per §11.4.115, the gate's BASELINE ratcheted to 0, and the ratchet's monotone-decreasing property preserved throughout (§11.4.227(A)). NOTE the reviewer's MINOR-1: a count-baseline absorbs a one-out-one-in swap, so remediation progress must be checked against the findings SET, not only the count. BOB-194 Bug In progress – WHAT: agent worktrees under .claude/worktrees/ are ephemeral copies pinned at arbitrary commits - never built, never shipped, deleted when the agent finishes. Pre-build gates that walk the tree with find(1) from the repo root descend into them anyway. MEASURED INSTANCE (2026-08-25) CM-GO-TOOLCHAIN-MATCHES-BUILDER FAILED the main build with exactly one finding, from .claude/worktrees/agent-afda1df906c537ab4 - a worktree 86 commits behind still carrying 'FROM golang:1.23-

ATM ID	Type	Status	Severity	Description
				alpine' against a go.mod long since at 1.26.2. The main tree was CORRECT througho Per §11.4.201(1) a false-positive refusal a FAIL-bluff of equal severity to a false pass: it blocks real work and teaches people to bypass the gate. FIXED FOR ONE GATE: '.claude' added to that gate's PRUNE_DIRS with a deterministic §1.1 pair - a stale mismatch inside .claude/worktrees must NOT fail a healthy tree, the SAME mismatch outside it MUST still fail, so the prune cannot widen into blindness (§11.4.201(6)) Mutation verified: reverting the prune failed 2 checks. SURVEY, measured not inferred: 8 13 pre_build gates walk the tree; only the fixed one prunes .claude. Gates using 'git ls-files' are structurally immune - worktrees are untracked. Empirically probed: check_cm_closure_seam_binds, check_cm_killpg_pgid_guard and check_cm_no_production_mutation_residue all rc=0 with ZERO worktree references - clean practice today. UNMEASURED, not clean: check_cm_test_mock_pid_explicit_int and check_cm_test_mock_pid_patched_when_real_p were not reached before the probe budget expired. HYPOTHESIS REFUTED, recorded so nobody re-walks it: an earlier revision of this item speculated that check_cm_plugin_count's 90s timeout was caused by walking worktrees. It is NOT. The gate's find is scoped to \$PLUGINS_DIR (check_cm_plugin_count.sh:224), so worktrees are outside its walk entirely. Its slowness is real - still running past 54s on a re-measure - but has a different, still-unidentified cause and belongs to its own item, not this one. ACCEPTANCE: every tree walking gate either prunes agent scratch or is proven immune by measurement, each with paired mutation. NOTE the asymmetry that bounds the risk: a stale worktree can only ADD findings to a security-relevant scan

ATM ID	Type	Status	Severity	Description
				(killpg, mutation-residue), never remove t - so the exposure is false refusal and erc trust, not a missed defect. B0B-195 In progress – **OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): TEACH THE SCANNER With NO – KEEP SIM105** The §11.4.252 fail-open scanner is FIXED to seecontextlib.suppress; ruff's SIM105 rule STAYS ENABLED in pyproject.toml. Disabling SIM105, and the belt-and-braces both-at-once option, were offered and NOT chosen. Rationale carried with the decision: fixing the gate makes SIM105 harmless, whereas disabling SIM105 alone would leave the gate blind to any suppress written by hand, inherited from a dependency's style, or already present in tree. REQUIRED BEHAVIOUR:with contextlib.suppress(...).andwith suppress(...) (thefrom contextlib import suppressbinding) wrapping a dangerous-combination call are DETECTED with the same severity and messag shape as the try/except/pass form; a narrowsuppress(SpecificError)around a NON- dangerous call must NOT fire, since a fals positive refusal is a §11.4.201(1) FAIL-bl of equal severity to a missed real one. CONSEQUENCE FOR THE NUMBER: once landed, t scanner's finding count stops being a flood over try/except shapes and becomes a genui census – any prior count citing completene was scoped to try/except only. This answer is recorded as consumer DATA per §11.4.35 it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settl call (§11.4.112(5) bounded-verdict discipl applied to decisions). --- prior item tex follows --- WHAT: the §11.4.252 fail-open scanner matches Try-handler shapes (A1)/(A contextlib.suppress is a With node, so a

ATM ID	Type	Status	Severity	Description
				swallow written that way is invisible. VERIFIED BY THE CONDUCTOR with a control needle through the same invocation path (§11.4.201(7)(b)), because a first attempt returned 0 for BOTH forms and was itself blind - the gate takes --root, not a positional, so a bare path exits on an unknown arg (§11.4.201(7)(c): the invocation path is part of the instrument). Correct r try/except/pass around <pre>os.unlink(user_supplied_path) -> 'FAIL - swallowed exception ... :5', the needle se contextlib.suppress(Exception) around the IDENTICAL call -> 'PASS - no swallowed- exception ... anti-patterns found'. Both combine untrusted input with an irreversib unlink; both swallow everything; one is caught and one is not. WHY IT IS WORSE THA PLAIN GAP: pyproject.toml enables ruff's S ruleset, and SIM105 is 'use contextlib.suppress instead of try-except- pass'. Our own linter therefore instructs authors to rewrite the form this gate CAN into the form it CANNOT. The two tools are pulling in opposite directions and the lin one runs more often. Every SIM105 autofix silently shrinks this gate's coverage whil the count goes down, which reads as progre CONSEQUENCE FOR THE NUMBER: the scanner's finding count is a FLOOR over try/except shapes, not a census of swallows. Any statement of the form 'N fail-open sites remain' must be read as 'N among try/excep shapes'. ACCEPTANCE: the scanner recognise contextlib.suppress (a With whose items ca contextlib.suppress with a broad exception type) and applies the same ≥2-capability test; a golden-TRUE fixture in suppress fo a golden-FALSE fixture where suppress wrap single-capability diagnostic emitter and m NOT fire; and an explicit decision on the SIM105 tension - either exempt the shapes this gate governs, or accept suppress and </pre>

ATM ID	Type	Status	Severity	Description
				teach the gate to read it. Discovered by t \$11.4.252 remediation agent when its own MINOR-3 fix required using contextlib.suppress at three diagnostic- emitter sites; those three uses are legitimate and justified in-source. BOB-196 Task Queued – WHAT: the CM PLUGIN-COUNT pre-build gate takes 189 SECCO to verify 8 documented counts. Measured 2026-08-25: exceeded a 90s probe budget (rc=124), then a bounded re-run completed 'rc=0 wall=189s' - it PASSES, it is simply slow. CAUSE, MEASURED NOT GUESSED: the mar loop at check_cm_plugin_count.sh:273-290 iterates EVERY LINE of every governed document and spawns a pipeline per line. P iteration: 'printf \\\n grep -oE \\\n wc -l \\\n -d' (4 processes) plus 'printf \\\n sed -n' = ~6. Governed docs are AGENTS.md 689 + CLAUDE.md 494 + docs/features/Status.md 67 1853 lines, so ~11,118 process spawns, measuring ~102 ms/line. Classic per-line- subprocess shell trap. THE IRONY WORTH PRESERVING, because it explains why nobody simplified it: the in-code comment shows ' \\n wc -l' was chosen DELIBERATELY over 'gr -c' after measuring the \$11.4.201(12) foot - on this host (ugrep 7.8.4) 'grep -coE' returned 3 at top level but 1 inside a 'se -euo pipefail' subshell, a context-depende reading. The correctness fix is right and must be kept; it is the PER-LINE applicati of it that costs the three minutes. HYPOTHESIS REFUTED, recorded so nobody re- walks it: this is NOT worktree traversal. gate's find is scoped to \$PLUGINS_DIR (:22 so the five agent worktrees are outside it walk. That guess was raised on BOB-194 and dead. LIKELY FIX (unverified, stated as a candidate not a conclusion): pre-filter wi ONE grep for lines containing 'CM-PLUGIN- COUNT:' before entering the loop, reducing ~1853 iterations to the ~8 lines that carr

ATM ID	Type	Status	Severity	Description
				marker, while leaving the per-line parsing logic - and its footgun-avoiding form - completely unchanged. WHY IT MATTERS beyond impatience: this gate is on the pre-build critical path, and CLAUDE.md leans on it as the mechanical authority for the three distinct plugin rosters (43 curated / 43 engines / 12 bootstrap) whose conflation was BOB-149. A gate slow enough to tempt anyone into skipping it protects nothing. ACCEPTANCE: runtime reduced with the count and the footgun-avoiding parse preserved byte-for-byte in behaviour, proven by the existing paired mutation still biting plus before/after wall-clock recorded. §11.4.6: nothing here questions the gate's VERDICTS only its runtime. BOB-197 Bug In progress – **OPERATOR DECISION (2026-08-26, §11.4.66): GENERATE THE TOKEN AND ARM IT** The conductor generates a 32 byte random token and writes BOBA_API_TOKEN into .env; the operator configures clients with it. DONE 2026-08-26: token generated secrets.token_hex(32), appended to .env with a comment naming BOB-197 and the routes.py env-gating fact, chmod 600, value shown to the operator ONCE and never written to any other file. Safety verified BEFORE writing (§11.4.30/§11.4.10): .env matched .gitignore:27 '*.env' and was untracked (needle-proven – the same match confirmed docker-compose.yml IS tracked, so the negative is real); a §9.2 pre-op backup was taken and is itself ignored; post-write re-check confirmed .env still ignored and absent from git status. Pre-store audit found the only tracked BOBA_API_TOKEN= occurrences are the docker-compose passthrough \$ {BOBA_API_TOKEN:-} and prose in the leak-audit challenge – no literal value has ever been committed. STILL OWED, and the item stays open until it lands: the BOOT-TIME INVARIANT (§11.4.254) that refuses to start

ATM ID	Type	Status	Severity	Description
				LAN-bound when the token is unset or empty. Without it the arming is a state fix, not a defect fix – §11.4.226(5) is explicit that a state-only repair cannot close a defect item. Options not chosen: operator sets it themselves with the invariant blocking until then; invariant warns first and blocks on a named date. Recorded as consumer DATA per §11.4.35 – the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. --- prior item text follows --- OPERATOR DECISION (2026-08-26): arm BOBA_API_TOKEN and keep 0.0.0.0, backed by BOOT-TIME invariant that refuses to start LAN-bound with the token unset. This item is that invariant. MEASURED FACT, not inference. download-proxy/src/api/routes.py:83 require_api_token() reads BOBA_API_TOKEN at request time and returns immediately when unset or empty. Its own docstring states it plainly: "BOBA_API_TOKEN unset/empty -> return (OPEN). This is the DEFAULT and preserves the current no-auth contract + dev workflow". docker-compose.yml:237 passes BOBA_API_TOKEN=\${BOBA_API_TOKEN:-}, so the container inherits empty unless the operator declares it. The operator .env does NOT declare it, needle-verified per 11.4.201(7)(b): the same match hit QBITTORRENT_DATA_DIR in the same file, the miss is a real absence and not a blind read. The qbittorrent-proxy container was RUNNING and LAN-bound at measurement time. Consequence: the ten routes the BOB-102 gateway resolves as auth-wired are open in practice on a default deploy. WHY A STATIC GATE CANNOT COVER THIS. The BOB-102 gate asserts static route WIRING - that a Depends(require_api_token) marker exists on handler, decorator or router. It cannot assert runtime ARMING, which depends on an env var read per request. Claiming the static

ATM ID	Type	Status	Severity	Description
				gate covers it would be exactly the bluff gate exists to prevent (11.4.262: machine evidence must match the layer it claims). This is a boot-time invariant class per 11.4.254. ACCEPTANCE: a boot-time check that, when the listener is LAN-bound (not loopback), refuses to start with a non-zero exit and a named cause if BOBA_API_TOKEN is unset or empty, so the open state becomes UNREACHABLE rather than the default. Paired 1.1 mutation: remove the check, prove the service starts LAN-bound-and-open. Golden- FALSE per 11.4.201(1): a loopback-bound listener with the token unset must NOT be refused, or the check becomes a false- positive refusal blocking legitimate dev work. 11.4.238 COVERAGE-ESCAPE NOTE: found by a subagent reading source during BOB-10 guard construction, NOT by the automated C regime - which is itself the defect class 11.4.238 names. The boot-time check IS the new automated check that would have caught it. BOB-199 Task Queued – **OPERATOR DECISION (2026-08-26, §11.4.66) FLAG NARROW SUPPRESS ONLY WITH AN IRREVERSIBLE CAPABILITY** The scanner flag a narrow contextlib.suppress ONLY when combined with an irreversible capability (delete / truncate / kill) – the shape that actually causes harm. Idiomatic narrow tolerances stay quiet, so the gate keeps its credibility and no false-positive storm trains readers to ignore it. Options not chosen: accept the asymmetry permanently with a header statement; flag all narrow suppress and absorb existing sites via a justified exemption list. MEASUREMENT CORRECTED TWICE and the second correction explains the first the original '37 narrow sites, all idiomatic was propagated by this conductor without verification, then re-measured by the authoring agent as OBSERVER CONTAMINATION (§11.4.201(10)) – 27 vendored third-party

ATM ID	Type	Status	Severity	Description
				sibling-agent worktree COPIES of the very tree being measured + 1 real = 37. The instrument was counting other agents' duplicates of its own subject. Reproducible figures, each with its scope stated: constitution repo 0 narrow with-suppress; boba DANGER_ROOTS 0 narrow with-suppress a 23 narrow except-pass; boba repo minus vendored 1 narrow with-suppress (a suppress(0SError)); narrow except-pass RETRACTED-AND-RESTATED 2026-08-26 per 11.4.201(9): the figure 97 previously recorded here reproduces under NO scope and is withdrawn. Measured replacements, each with its scope and both independently reproduced: 34 narrow except-pass git-tracked first-party (canonical); 50 narrow except- pass on-disk minus scratch dirs (the 'minus- vendored' scope this sentence states). Brackets that explain the bad number: 87 A except-pass git-tracked and 113 ALL-except- pass on-disk-minus-scratch straddle 97, so was a class-scope conflation (narrow-vs-all) not a corpus-scope one. The 35 cited later this record is the same measurement at git- tracked scope and agrees with 34 within one site. The 'every one idiomatic' claim was also false – the except-pass census contained SystemExit x2 (tests/unit/ test_plugin_torrentscsv.py:305,328), which not a benign tolerance. CORRECTION 2026-08-26: a 'KeyboardInterrupt x1' previously written in this sentence was itself unsourced and is WITHDRAWN – it reproduces at NEITHER stated scope. Independent AST count over git-tracked *.p ZERO genuine 'except KeyboardInterrupt: pass'. The only genuine instance in the checkout is .worktrees/ci-split-workflows/ tests/unit/test_main.py:63 – a sibling-age scratch worktree, i.e. the exact observer- contamination class (11.4.201(10)) this ve record teaches to exclude – and the only

ATM ID	Type	Status	Severity	Description
				main-tree textual match (tests/unit/test_graceful_shutdown.py:165) sits inside triple-quoted string literal, a carrier no code (11.4.201(7)(a)). Recorded because the error is instructive: it was introduced BY the retraction that removed the 97, so a correction is not exempt from the discipline it applies. Recorded as consumer DATA per §11.4.35 – the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. --- prior item text follows --- RESIDUAL ASYMMETRY surfaced by the BOB-195 fix, reported rather than silently closed. After teaching the scanner With nodes, a BROAD suppress (Exception / BaseException) is detected with the same severity as try/except/pass. A NARROW suppress (suppress(FileNotFoundError)) is - but the semantically identical narrow try except FileNotFoundError: pass IS flagged. ruff SIM105 rewriting a narrow handler still moves that site out of the gate's scope, a smaller version of the exact mechanism BOB-195 was filed to close. WHY IT WAS NOT CLOSED IN THAT PASS, with the measurement that decided it: the authoring agent reported 37 narrow suppress sites, all idiomatic. CORRECTION (2026-08-26, independent review) that figure DOES NOT REPRODUCE and this conductor propagated it into this item as fact without verifying - the error is mine, not the reviewer's. Measured: the constitution repo has 0 narrowwith suppress(X)sites; boba first-party has exactly 1 (a suppress(OSError)). The nearest corpus match is 35 narrowexcept X: passhandlers in boba first-party - a DIFFERENT construct - and their class cens (OSError x7, ImportError x7, BrokenPipeError x3, RuntimeError x3, ValueError x2, FileNotFoundError x2, IndexError x2, SystemExit x2) contradicts 'every one

ATM ID	Type	Status	Severity	Description
				idiomatic': a bareSystemExit: passis not a benign tolerance. The two populations must not be conflated, and the distinction is the whole substance of this item. The false-positive-storm argument therefore rests on the 35 narrow except-handlers, not on 37 suppress sites, and its strength should be re-judged on that basis, which under 11.4.201(1) is a FAIL-bluff of equal severity to the gap it would close, and worse in practice because it trains readers to ignore the gate. The subagent recorded the asymmetry in the gate header as a known gap rather than shipping the storm. That was the right call and is why this is a separate tracked item rather than an unfinished one. THE DECISION IS THE OPERATOR'S, mirroring BOB-195 itself (a) accept the asymmetry permanently, with the gate header stating it so no reader mistakes the count for a census; (b) flag narrow suppress ONLY when combined with an irreversible capability (delete / truncate / kill), catching the shape that actually matters and leaving the 37 idiomatic sites quiet; (c) flag all narrow suppress and absorb the 37 via a justified exemption like the LAN-route guard uses. RELATED FAULTS worth carrying: the five newly-visible production sites in download-proxy/src/merge_service/search.py (1294, 1303, 1322, 1329, 1333) wrap proc.kill / os.killpg / proc.wait in the BOB-126 cleanup path. The conductor verified the 11.4.263 pgid guard present and correct at both killpg sites - _pid and _pgid each checked isinstance(int) and > 1 before the syscall, with the BOB-1 forensic reasoning inline - so those sites are true-by-the-scanner's-definition but S in fact, and want a justified exemption error rather than a code change. BOB-200 E Queued Medium WHAT: scripts/gates/cm_dangerous_combination_fail_closed.sh builds its scan file list with find(1). If

ATM ID	Type	Status	Severity	Description
				find itself DIES (permission error, resource exhaustion, interrupted), the gate receives an EMPTY file list and interprets that as 'files in scope' -> honest topology SKIP -> exit 0. The failure IS loud on stderr but SILENT in the exit code, so any caller gate on exit status reads a dead instrument as clean corpus. WHY IT MATTERS: this is a textbook 11.4.201(6) FALSE-NULL -- a blind instrument and a clean artifact return the identical quiet zero. It is also a 11.4.25 fail-OPEN on exactly the 'cannot enumerate the corpus' condition where the gate's own stated discipline is to fail CLOSED. A gate that cannot see must refuse, not pass. AFFECTED SCOPE: constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh (the find invocation and the empty-list branch) PRE-EXISTING -- NOT introduced by the BOB- change; found by the independent round-2 reviewer while reviewing that change and explicitly scoped OUT of its remediation. REPRODUCTION: make find(1) fail during the gate run (unreadable scan root, or a find stub returning non-zero with empty stdout) and observe the gate exit 0 with a topology SKIP message, indistinguishable from a genuinely empty corpus. ACCEPTANCE: (1) the gate distinguishes 'find succeeded and found zero files' from 'find failed' -- check find's exit status, not only its output; (2) a failed enumeration is a FINDING (non-zero exit) naming the unresolved precondition, never a SKIP; (3) a genuinely empty scan must still SKIPs honestly at exit 0 -- the 11.4.201(1) golden-FALSE guard, so the fix does not become a false-positive refusal; paired 1.1 mutation: restore the swallow-find-failure behaviour and the new fixture MUST fail. BOB-211 Bug Queued minor STATUS: LATENT on this host (measured: a six declared locations sit on ONE btrfs mount, fully ownership-expressive) and

ATM ID	Type	Status	Severity	Description
				OPERATOR-GATED for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being ex or NTFS is entirely ordinary. TWO RESIDUAL (1) REPAIR-LAYER DIAGNOSIS. On a uid-flattening mount whose flattened uid is NO the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 bo refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstype-aware – so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, NOT proven: the shim used to verify BOB-206 models the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dmask, so the preserve_mode: true / 600 contract on .env and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak to mode check cannot see. WHY OPERATOR-GATED genuine end-to-end RED needs a real uid-flattening mount. The privilege-free route bindfs -u <uid> (FUSE; fusermount IS present and user_allow_other IS set) – but bindfs ABSENT on this host, as are mkfs.vfat and losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was attempted (§11.4.21 – high-blast-radius host mutation is operator-gated). ALSO RECORDED: NO test anywhere exercises a uid-flattening filesystem – vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure 0 across the three ownership test files, needle-proven (probe_location = 3 hits through the same

ATM ID	Type	Status	Severity	Description
				instrument). ACCEPTANCE: (1) operator decides whether to install bindfs or provide a privileged loopback so the RED can be built; (2) if verified, the repair path generates fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mounts and refuses, or documents honestly that it cannot be enforced there. DISCOVERY CHANNEL (§11.4.238): retained residual from the BOB-206 verification stream. BOB-213 Bug Queued critical WHAT: the §11.4.252 fail-closed gate's extension list omits shell entirely. Measured verbatim by the conductor at constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh:45 <pre>- exts="\${DANGEROUS_COMBO_EXT:-py go rs cc cpp h hpp java cs js ts jsx tsx php rb}</pre> No sh. No bash. THE CONSEQUENCE, MEASURED scripts/ IS a declared DANGER_ROOT (script pre_build_verification.sh:1511). It contains 71 tracked .sh files and 3 .py files. So 7 of 74 source files in a root the gate is explicitly pointed at are SILENTLY INVISIBLE – while invariant 39's driver prints "no fail-open anti-pattern across N first-part source root(s)". Project-wide there are 19 tracked .sh files (negative control *.zzz 0, instrument proven seeing). WHY THIS IS THE WORST OF THE THREE HOLES: BOB-205 is a scope hole (root never looked at). BOB-191 is a matcher hole (Go enumerated but unanalysable). THIS one is worse than either because the root IS declared, IS scanned, counted as covered, and 96% of what is in it was never examined. The count in the driver's own summary is an under-count presented as a census. AND SHELL IS THE HIGHEST-RISK LANGUAGE HERE, not the lowest: the project's orchestration, credential handling, contains control and gates are shell. start.sh alone is 1288 LOC and is, per CLAUDE.md, THE orchestrator. Fail-open in shell is also

ATM ID	Type	Status	Severity	Description
				unusually easy to write – a barecmd tru an unchecked\$?, aset+eregion, a swallowed2 dev/null – and §11.4.67(6) already records one live instance of exactly this class (a bare exe redirection silencing an interactive shell for its lifetime). CORRECTION TO A PREVIOUSLY-STATED ACCEPTANCE CRITERION (§11.4.6): I asserted, when filing BOB-205 that "the honest-blindness path (§11.4.3 SKIP-with-reason, which the gate already implements correctly for unenumerated extensions) is preserved". THAT IS WRONG. gate's honest SKIP-with-reason exists for PYTHON ARM's degradations, NOT for extensi absent from the ext list. An unenumerated extension is a SILENT false-null, not an honest skip. The distinction is load-beari and I stated it backwards. NOTE ON LINE NUMBERS: the BOB-205 stream cited the ext list at :318; the conductor measured it at :457. Both are correct at their read times a sibling stream (BOB-195 r7) is actively editing this file. Re-derive before acting ACCEPTANCE: (1) either shell is added to t ext list WITH an arm that can actually analyse it, or unenumerated extensions produce an explicit UNANALYSED verdict per file – never a silent pass (this is the sa analyser-registry fix BOB-191 needs, and doing it once covers both); (2) the driver summary reports files ANALYSED, not files present, so an under-count cannot masquera as a census; (3) a RED planting a shell fa open under scripts/ and asserting it is SE (4) golden-FALSE per §11.4.201(1) – a correctly fail-CLOSED shell guard must NOT flagged; §11.4.67's own brace-scoped exec form is the natural fixture. COMPOSES: BOB-191 (matcher hole – same primitive: classify by local shape, never consult semantic role; and the analyser-registry f closes both) and BOB-205 (scope hole). Per

ATM ID	Type	Status	Severity	Description
				§11.4.250 these are three symptoms of one primitive defect; per §11.4.214 they stay three items because the fixes differ. DISCOVERY CHANNEL (§11.4.238): found by the BOB-205 verification stream, confirmed independently by the conductor. Not by the automated QA regime. BOB-214 Bug Queued critical WHAT: DANGER_ROOTS = (download-proxy/src plugins scripts qBitTorrent-go frontend/src) at scripts/pre_build_verification.sh:1511. The REPOSITORY ROOT itself is not among them (conductor-verified: no "." entry). Everything living at the top level is scanned by no arm. THE LIVE HIT: webui-bridge.py tracked at root (conductor-verified), 466 LOC, a live HTTP service on port 7188 (BaseHTTPRequestHandler:85, do_GET/do_POST:92-97, request path read at :103, outbound urlopen at :273, environment credentials read at :55-74). The gate REPO A REAL HIT AT :295 when pointed at it directly. This is not a hypothetical gap: existing gate, unmodified, finds a genuine defect in this file the moment scope reaches it. ALSO UNSCANNED AT ROOT: 13 first-party shell scripts including start.sh (1288 LOC per CLAUDE.md the project's orchestrator and the sole sanctioned container-control entry point), stop.sh, ci.sh, install-plugin.sh, and the credential-handling init-qbit-password.sh / fix-qbit-password.sh. (These are additionally invisible for the separate LANGUAGE-hole reason – shell is not in the gate's ext list – so root files get missed twice over, by scope AND by extension.) THIS SCALE THIS SITS IN: 266 files in scope / 500 out of scope – 66% of the gate-visible first-party corpus is never scanned. Other unscanned roots measured: tests/ (324 files excluded-by-intent but UNDECLARED – an undeclared exclusion is exactly what §11.4.224(E) fences against), extension/

ATM ID	Type	Status	Severity	Description
				(108, shipped browser extension), docs/ (5 challenges/ (18), tools/ (1, and it CONCEALS 3 REAL HITS at plugin_update_automation.py:189,198,215), frontend/e2e + 2 configs (5). SO THE SCORE HOLE CONCEALS AT LEAST 4 REAL HITS the gate itself finds when pointed at them (1 in webui-bridge.py, 3 in tools/). Invariant 3 reported count is an under-count, not a census. ACCEPTANCE: (1) repository root and tools/ are scanned, or explicitly fenced with a stated reason per §11.4.224(E) – silence is not an exclusion; (2) the 4 concealed hits are triaged (each is either a real defect to fix or a false positive to fix in the gate – both are findings); (3) tests/ (324 files) gets an OPERATOR decision per §11.4.66 – production-only is defensible but must be DECLARED; (4) a RED proving a root-level fail-open is seen; (5) golden-FALSE proving genuine build artefacts and vendored trees stay excluded. FIX DIRECTION (measured by the BOB-205 stream, not guessed): a §11.4.224(E) manifest is feasible but the source of truth must be chosen carefully – docker-compose. build contexts MISS plugins/, frontend/src/cmd/boba-ctl and webui-bridge.py; language markers (go.mod/package.json) MISS plugins/ and webui-bridge.py. Only derive-from-git-tracked-source-extensions minus a declared §11.4.224(E) exclusion fence reaches every gap. Note the derivation must be built either way – if the operator prefers keeping a hash list, the omission-guard that audits it is the SAME computation; the only question is whether it drives the scan or audits the list. DISCOVERY CHANNEL (§11.4.238): four by the BOB-205 verification stream while enumerating the full gap list – the item it was verifying named only one missing root. Not by the automated QA regime. === CORRECTION 2026-08-26 – THE "4 CONCEALED REAL HITS" CLAIM IN THIS ITEM IS REFUTED (§11.4.

ATM ID	Type	Status	Severity	Description
				=== When I filed this item I wrote that the scope hole "conceals at least 4 real hits that the gate itself finds when pointed at them". A dedicated triage stream ran the UNMODIFIED gate against scratch copies, reproduced all four hits exactly, and triaged them: **4 of them are FALSE POSITIVES.** The instrument was control-needle-proven in BOTH directions first (fired on a planted except: pass, stay silent on a clean log-and-reraise), so this is evidence, not a null. All four are the SAME detector class – shape (A2) "silent default return" – which is precisely the BOB-189 false-positive shape: the returned literal IS the function's designed refusal value, consumed by a caller that checks it. - webui-bridge.py:295 (_is_root_liveness_probe): False -> send_error(502) at :438. The except DENIES the lenient 200 path. Fail-CLOSED. Combined ZERO of the six §11.4.252 capabilities – a pure classifier over self.path/self.command - tools/...:189 (_download_url): None -> : handles it explicitly (warn + continue), r write. Handler BOUNDED to (URLError, HTTPError); anything else propagates. - tools/...:198 (_extract_version): "unknown gates nothing – the update decision is the HASH compare at :116. (It IS a bareexcept:though, so Ctrl-C is swallowed separate signal-handling defect, filed with the README item.) - tools/...:215 (_validate_plugin): False -> :151 returns BEFORE the write at :165. This IS the refusal. Canonical BOB-189 shape. WHAT TH DOES AND DOES NOT CHANGE. The scope hole itself is UNCHANGED and still real – the repository root and tools/ are in no DANGER_ROOT, verified verbatim at scripts/pre_build_verification.sh:1511. What change is the ARGUMENT: I justified this item partially on "it hides four known defects", and that justification is wrong. The correct

ATM ID	Type	Status	Severity	Description
				justification is narrower and does not depend on any current hit: webui-bridge.py is a LIVE, LAN-BOUND HTTP service (see below) handling request paths, outbound calls, and environment credentials – a surface that belongs in scope on its capabilities, whether or not it currently contains a defect. NE FACT RAISING THE STAKES – 7188 IS LAN-REACHABLE. webui-bridge.py:447 binds ThreadingHTTPServer("", BRIDGE_PORT) – conductor-verified. The empty host string INADDR_ANY, so port 7188 listens on ALL INTERFACES, not loopback. Neither localhost only nor internet-exposed (internet only if forwarded). No severity is assigned to :29 because it is fail-closed – but this is the argument for putting the root in scope: a live LAN-bound service should be scanned on principle, not after a defect is found in it. webui-bridge.py IS LIVE, DECISIVELY (§11.4.124 never engages – do NOT treat it superseded): 142 tracked files reference it including config/served_ports.yaml and the credential-leak-audit challenge; download-proxy actively probes it via bridge_health at download-proxy/src/api/__init__.py:198; newest of 12 commits is a real functional (6e3da73, 2026-06-14). Critically, served_ports.yaml:66-69 records that the G webui-bridge is a SEPARATE binary the qbittorrent-proxy-go container never starts so NOTHING ELSE BINDS 7188 and this Python file is the only thing serving that port. tools/ RECOMMENDATION (§11.4.224(E)): FENC IT OUT, DECLARED – exclusion class "non-shipping developer utility". Evidence: 2 tracked files, neither gitignored; the script is invoked by NOTHING (control-needle-prov 91 needle hits, 0 negative control – referenced only by its own README, its own docstring, and a tracker note; zero hits across *.sh/*.yaml/*.py/Makefile outside tools/); its README self-declares "not

ATM ID	Type	Status	Severity	Description
				invoked by the normal start/stop flow"; la functional commit 2026-04-12. Because it i FIRST-PARTY, §11.4.224(E) requires the exclusion carry a tracked §11.4.197 item – and the two REAL defects found in it must tracked regardless of scope, since a hand- --update still executes them. BOB-216 Bug Queued major WHAT: three carrier classes fire in the gate's PRIMARY mode, r only its degraded text fallback. Measured ast_rc=1 AND text_rc=1 (both modes report hit): (a) a COMMENT or DOCSTRING quoting credential anti-pattern is reported as a L credential default – both spellings; (b) a // comment or a string constant holdingt { x(); } catch (e) { }` is reported as an empty catch, once per carrier line. Shape (b), and th family half of shape (a), are LANGUAGE- AGNOSTIC GREPS with NO structural counterpart in any mode — so there is no pa to be immune. WHY IT MATTERS: the gate's header carried a blanket claim that “A parser immune to both by construction”. That is OVI BROAD and now corrected in place, scoped to the Python Try/With shapes where it is actual true. Everything else — every non-Python extension, and the credential/empty-catch tex matchers even on Python — has no AST arm all, so the immunity never applied there. A §11.4.201(1) false refusal in the PRIMARY mo is materially worse than one in a fallback nobody expects to be exact: it refuses provab healthy code on the path the gate is trusted o THE CONTROL NEEDLE THAT SHARPENS IT (recorded so the next reader inherits the measurement): a #-COMMENTED import doe NOT poison the licensing table (text=0), becau those regexes are line-anchored. So the blindness is to STRINGS specifically, not to no code generally. That distinction is what make the comment-strip fix tractable for one class not for the others. WHY IT WAS NOT FIXED I THE ROUND THAT FOUND IT (§11.4.6 honest

ATM ID	Type	Status	Severity	Description
				boundary, and this is the right call): closing these needs a PER-LANGUAGE comment-and-string model across every configured extension. That model's failure direction is the UNDER-reporting one — a mis-parsed string region silently swallows real violations for the remainder of the file. The same reasoning makes the round DECLINE the triple-quote fence counter for OVER-1 while ACCEPTING the comment strip for OVER-A: the comment strip ambiguity resolves toward KEEPING text (proven by three control needles — real violation + comment, # inside a string, escaped quote before # — all 1/1), whereas a fence counter's ambiguity resolves toward DROPPING text. Direction of failure, not difficulty, is the discriminator. Building a full lexer here would import the under-reporting failure mode this gate refuses everywhere else. OPERATOR DECISION (§11.4.66 / §11.4.197): whether to build per-language comment-and-string model is a consumer call, not a default this gate should pick unilaterally. Options: (a) accept the over-reports and DISCLOSE them per class (what the round did — a new MODE-INDEPENDENT CARRIERS section now enumerates all three) build the models per language and own the under-report risk with a fixture per language (c) narrow the language-agnostic greps so they only fire where a structural arm exists, trading coverage for precision. RELATION TO SIBLING (§11.4.214 distinct-but-similar, deliberately not merged): BOB-189 is the gate flagging a fail-CLOSED SSRF guard — a semantic-role miss in the PYTHON arm. This is a CARRIER miss (comment/string read as code) that is MODE-INDEPENDENT. BOB-213 is a language absent from the ext list entirely. All three share the primitive §11.4.250 named in BOB-191: classified by local syntactic shape, never consult semantic role — but the fixes differ, so the items stay separate. ACCEPTANCE: (1) the operator decision above is taken and recorded; (2) whichever p

ATM ID	Type	Status	Severity	Description
				is chosen, each of the three classes has a fixt proving current behaviour, so a future chang cannot silently alter it; (3) if (b) is chosen, a golden-FALSE per language proving a REAL violation adjacent to a carrier is still caught – the under-report guard, which is the whole r DISCOVERY CHANNEL (§11.4.238): found by t BOB-195 round-7 remediation stream while closing a different finding, NOT by the automated QA regime and NOT by the independent reviewer that audited the same in round 6. WHAT, conductor-verified: docs/guides/track credentials.md is TRACKED. Its §11.4.65- mandated export twins docs/guides/tracker- credentials.html and .pdf both EXIST ON DIS are NOT tracked, and ARE ignored — matche by the .gitignore <i>credentials</i> deny-all at :31. The .md is rescued by an explicit allowlist en at :38; nobody added entries for the twins. W IT IS A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in- scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.2 requires every §11.4.65-scope doc to be reachable from the README. Two artifacts th exist locally but can never be committed are, from any fresh clone, ABSENT — so every clo of this repository has a tracker-credentials gu with no exports, while the authoring host loo complete. The divergence is invisible on the machine that would notice it. WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL of BOB-212 class. git status never listed the twin so no gate and no author had a signal. The bl instrument and the clean tree return the identical quiet zero. ANOTHER INSTANCE OF THE SAME MECHANISM, filed here rather th separately because it is one fix: docs/qa/ BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and is therefore uncommitted §11.4.83 QA evidence
BOB-219	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				SIBLINGS IN THE SAME DIRECTORY — user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log — ARE tracked, saved only by the robust glob negation !docs/qa/**/**.log. That one file is lost purely because it is . rather than .log. The pattern is exact: robust negations hold; per-file and per-extension rescues leak. ACCEPTANCE: (1) both twins become trackable and are committed alongside their .md; (2) the BOB-124 .txt evidence likewise; (3) whichever BOB-212 fix direction is chosen MUST cover these — a fix that closes the future to swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check to a tracked .md in §11.4.65 scope has COMMITTABLE twins, so this class cannot recover silently. DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep — a only on its SECOND pass. The first pass filtered by a source-extension set that omitted .pdf, which hid this finding entirely; the agent re-ran with no extension filter over all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowlist is itself a false-null generator which is the same shape as the defect being investigated. WHAT, both conductor-verified with control needles: CM-SCRIPT-DOCS-SYNC occurrences scripts/pre_build_verification.sh = 0 (control needle CM-MARKDOWN-EXPORT-SYNC = 6, so the instrument sees). §11.4.44 revision-header enforcement likewise 0 hits (control needle 3). Neither §11.4.18's companion-doc mandate nor §11.4.44's revision header is enforced by any invariant. THE PERVERSE CONSEQUENCE, MEASURED: a new script with NO companion doc passes the gate. Writing the companion doc that §11.4.18 MANDATES creates docs/scripts/.md, which invariant 16 (CM-
BOB-223	Bug	Queued	major	

ATM ID	Type	Status	Severity	Description
				MARKDOWN-EXPORT-SYNC) then requires to have .html and .pdf twins — and their absence is a BLOCKING failure. This session hit it exactly at docs/scripts/test_gitignore_swallow_is_loud. {html,pdf} are missing and are two of invariant 16's six violations. So the gate's incentive gradient points AWAY from the constitution: complying with §11.4.18 is punished, ignoring it is free. That is worse than an ungated rule — it is a rule the machinery actively discourages. §11.4.227 IS THE GOVERNING ANCHOR: a named gate that exists only in prose is gate debt and the ledger of unimplemented CM-* names must be monotone-decreasing. CM-SCRIPT-DOCS-SYNC is named in §11.4.18's own text and implemented nowhere — one concrete row covers exactly the 58%-unimplemented population §11.4.227 was minted to shrink. ACCEPTANCE (1) either CM-SCRIPT-DOCS-SYNC is implemented, or it is registered as a deferral pointing at a tracked §11.4.197 item — silent absence is what §11.4.227 forbids; (2) same for §11.4.44's header check; (3) whichever is chosen the incentive inversion is closed: it must never be cheaper to skip a mandated doc than to write one — if invariant 16 will demand twins, the doc-creation path must produce them, or invariant 16 must scope out docs/scripts until it can; (4) a paired §1.1 mutation per gate implemented. HONEST NOTE ON SCOPE: this item does not argue §11.4.18 should be enforced immediately — that is an operator call about gate debt priority (§11.4.66). It argues the CURRENT state is incoherent: an unenforced mandate whose observance triggers a difference in gate's failure. Either enforce both ends or neither. DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight while explaining why a new file broke invariant 16. Not by the automated QA regime. WHAT: ownership_repair walks a declared row and repairs items whose uid is not the operator's.
BOB-226	Task	Queued	major	

ATM ID	Type	Status	Severity	Description
				The unit suite seeds a foreign uid only onto FILES and SYMLINKS; interior DIRECTORIES stay operator-owned (case 22 seeds only a foreign declared ROOT, on the failure path). Production first-start repairs exactly the untested shape. MANIFEST: tests/unit/test_ownership_repair.sh seed_tree/seed_wro scripts/ownership_repair.sh walk at :994. REE seed a tree whose interior directories carry u 100000 via podman unshare, run the repair, observe no automated assertion covers the outcome. WHY UNTESTED: the unprivileged harness cannot create symlinks inside a directory it no longer owns, which cases 8/9/ require. The DECLARED GAPS note cross-references tests/ownership/test_container_writes_owned_files.py, but tha covers the CREATION side (FR-002), not the repair-side walk. ACCEPTANCE: an integration layer test (where the unprivileged-harness constraint does not bind) that seeds foreign-owned interior directories, runs the repair, a asserts post-state ownership plus mode preservation. Surfaced by the BOB-207 independent review 2026-08-27. WHAT: §11.4.212 makes the main README the canonical entry point for ALL project documentation, with no §11.4.65-scope doc reachable by no link path. docs/scripts/check_cm_lan_routes_authenticated.md is no reachable from README.md. MEASURED 2026-08-27: grep -c for the guide name in README.md returns 0, control-neededled again a doc README does link (CONTINUATION returns 1), so the instrument is not blind. ACCEPTANCE: README links the guide directly or transitively, and the doc-link generator cov docs/scripts/ so the next such guide cannot la orphaned. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27.
BOB-228	Task	Queued	minor	
BOB-231	Task	Queued	Major	

ATM ID	Type	Status	Severity	Description
				BOB-227 measured 3 of 4 artifacts of the CM-LAN-ROUTES-AUTHENTICATED gate untracked in git; criterion 1 (commit them) was already resolved by an unrelated prior commit before this session, closed 2026-09-23. Criterion 2 was NOT addressed: ‘a gate asserting that every executable a pre-build invariant invokes is tracked’ – a general mechanism preventing the whole CLASS of defect (a pre-build gate’s own implementation files shipping untracked, invisible to a fresh clone, no committed base for round-over-round diffs) from recurring for ANY future gate, not merely this one. ACCEPTANCE: (1) enumerate every file path scripts/pre_build_verification.sh invokes (via bash/timeout/python3 calls to scripts under scripts/pre_build/, plus every tests/pre_build/ and tests/hooks/.sh it runs) – likely via a static grep/parse of pre_build_verification.sh itself, a runtime trace; (2) assert every one of those paths is git-tracked (git ls-files –error-unmatch); (3) wire this as a new pre-build invariant so a future untracked gate implementation is caught immediately, not discovered independently weeks later; (4) a RED test creating an untracked fake gate-invocation target and asserting the new check fails on it, GREEN once the mechanism exists and the fake target is either removed or tracked.